

A PUF-Rooted Post-Quantum Authentication Protocol for Pseudonymous UAV Remote Identification

HALA M. HAMADEH¹, AND ABDALLAH M. ALMOMANI²

¹CyberSecurity Department, Jordan University of Science and Technology, Jordan (e-mail: hmhamadeh@just.edu.jo)

²Aerospace Engineering Department, Jordan University of Science and Technology, Jordan, (e-mail: amalmomani@just.edu.jo)

Corresponding author: Hala M. Hamadeh (e-mail: hmhamadeh@just.edu.jo).

ABSTRACT Remote Identification (Remote ID) improves accountability for unmanned aerial vehicles (UAVs), but persistent plaintext identifiers enable unauthorized tracking and can be copied for spoofing or replay. This paper presents a PUF-rooted post-quantum authentication protocol that separates routine broadcast from authorized proof. During enrollment or renewal, the UAV acquires a majority-voted Ring-Oscillator Physical Unclonable Function (RO-PUF) response vector, reconstructs a 256-bit master secret with a syndrome-based fuzzy extractor, and derives independent signing and Merkle secrets. The signing secret deterministically regenerates the UAV's Module-Lattice-Based Digital Signature Algorithm (ML-DSA) identity. The Merkle secret generates commitments over interval pseudonyms. A Trusted Authority signs each root and distributes scoped authorization records without revealing PUF-derived seeds, future pseudonyms, private keys, or operator identity. The construction defines two deployment profiles. The pseudonymous Specific Session ID profile carries a leaf-derived 128-bit pseudonym in an ASTM-mapped identifier field only where the applicable authority and accepted means of compliance permit that identifier semantics. The supplementary-authentication profile leaves the jurisdiction-required Remote ID identifier and mandatory message set unchanged and provides the PUF-rooted proof only through an authenticated unicast channel. An authorized verifier establishes a post-quantum session through the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), receives a Merkle membership proof, and verifies an ML-DSA signature bound to the observed identifier, authorized root, verifier request, nonce, timestamp, and session ciphertext. The protocol discloses no interval secret or reusable lower-assurance credential and does not require a live Trusted Authority connection before each flight. The evaluation uses a circuit–software co-simulation: HSpice generates response matrices for a modeled 45 nm RO-PUF, and a Raspberry Pi 5 performs majority voting, fuzzy extraction, Merkle processing, and post-quantum operations. The resulting PUF evidence is specific to the adopted simulation model and does not establish fabricated-silicon entropy, aging behavior, physical-device environmental reliability, or resistance to physical attacks. The design establishes standards mapping and message-size compatibility rather than regulatory certification or DRIP broadcast-authentication conformance. ML-KEM-768 and ML-DSA-65 provide NIST category-3 security for key establishment and signatures, respectively; the complete protocol is not assigned a uniform category because its other guarantees are separately bounded by a 384-bit collision-sensitive hash, a 128-bit operational pseudonym, a simulation-derived response-entropy estimate, and a fuzzy-extractor statistical-distance target of 2^{-64} .

INDEX TERMS UAV Remote Identification, Pseudonymous Authentication, Post-Quantum Cryptography, Hardware Security, PUF

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) now operate in airspace where identification failure has real consequences: inspection flights near critical infrastructure, emergency-response

missions, and border or airport-adjacent operations all need a reliable way to distinguish legitimate UAVs from spoofed, misconfigured, or unauthorized platforms [1], [2]. Remote Identification (Remote ID) was introduced to provide that ac-

countability: a Remote ID transmitter periodically broadcasts identity and telemetry so nearby receivers can determine which UAV is operating in a given area.

The regulatory model is built around open broadcast, however. In the United States, 14 CFR Part 89 specifies the permitted identity forms, mandatory message elements, minimum performance requirements, and broadcast cadence for standard Remote ID aircraft and broadcast modules [3], [4]. ASTM F3411 and F3586 provide message and means-of-compliance specifications, while an FAA-accepted Means of Compliance and a manufacturer Declaration of Compliance remain separate regulatory steps [5]–[7]. In the European Union, Delegated Regulation (EU) 2019/945 addresses UAS and direct-Remote-ID product requirements, whereas Implementing Regulation (EU) 2019/947 governs operations [8], [9]. Japan maintains its own Remote ID device and application requirements [10]. These regimes use open Bluetooth and Wi-Fi broadcast, creating a privacy and security tension: a plaintext, long-lived identifier lets any nearby receiver follow a UAV across time, reconstruct its route, and locate its operator, whether or not that receiver has any legal or operational need to do so. Authorities need traceability under authorized conditions without letting ordinary observers turn the channel into a persistent tracking feed, and a practical authentication protocol must preserve that separation without treating field capacity as proof of compliance.

Authentication adds a second constraint. A broadcast identifier short enough for Remote ID payloads is easy to copy, replay, or spoof unless it can be verified, and classical public-key signatures face payload-size limits and the growing urgency of post-quantum migration: Shor's algorithm threatens the RSA and elliptic-curve schemes most authentication designs still use [11], and national cybersecurity agencies now treat that migration as urgent, not speculative [12], [13]. NIST's standardized post-quantum primitives [14]–[16] solve the cryptographic problem but not the size problem: their keys and signatures are far larger than a Remote ID broadcast field can carry, so a design that places post-quantum signatures directly in every broadcast message is impractical.

Existing privacy-preserving Remote ID protocols close part of this gap. ARID and A2RID provide anonymous broadcast, but ARID needs a live trusted-authority connection to resolve each pseudonym and A2RID's anonymous message-signing does not fit Remote ID's short broadcast frames [17], [18]; neither offers quantum resistance. pqRID separates identification from post-quantum authentication, achieving compact pseudonymous broadcast with stronger proof on a different channel [19], but its certificates and cryptographic material remain in nonvolatile storage and are therefore available to passive memory extraction. PUF-based UAV authentication schemes solve that exposure problem but target Internet-of-Drones authentication rather than standards-constrained Remote ID broadcasting [20]–[22]. DRIP defines requirements, architecture, identifier semantics, and paginated ASTM Authentication Message formats for trustworthy Broadcast Remote ID [23]–[26]; its DET identifiers and broadcast-authentication

formats are distinct from the rotating pseudonyms and unicast ML-DSA proof used here. No single protocol combines compact pseudonymous operation, hardware-derived identity, and post-quantum verifier authentication without requiring a live Trusted Authority connection before every flight.

This paper proposes a PUF-rooted Remote ID authentication protocol built around that gap. A Merkle root acts as an authorization commitment, while each leaf defines one interval pseudonym. Both are derived from an RO-PUF and fuzzy extractor rather than read from stored credentials. The Trusted Authority signs each root and distributes it to authorized verifiers by scope, so the UAV does not require a live authority connection before flight. Where an accepted Session-ID or Specific Session ID profile permits a rotating identifier, the UAV broadcasts the current pseudonym through the standards-mapped identity field. Otherwise, the jurisdiction-required Remote ID identifier and message set remain unchanged, and the proposed mechanism operates as supplementary authorized-verifier authentication. When proof is required, the verifier establishes an ML-KEM session and receives a Merkle path together with a transcript-bound ML-DSA signature. The post-quantum signature and Merkle proof remain outside the routine broadcast frame [14], [15].

Set against pqRID, compact pseudonymous broadcast and on-demand post-quantum proof are already established. The contribution here is the PUF-derived credential architecture beneath those functions: the signing identity, Merkle-root secrets, and interval pseudonyms are regenerated from an RO-PUF response rather than retained as credentials in non-volatile memory. The protocol-level protection is limited to passive storage disclosure; active control of an intact captured UAV can invoke the same reconstruction process used by legitimate firmware. The empirical study represents the RO-PUF interface through HSpice-generated response matrices and therefore evaluates architectural feasibility rather than a fabricated hardware root.

The main contributions of this paper are as follows.

- 1) **A PUF-derived credential architecture for Remote ID authentication.** The UAV's signing identity and Merkle-root secrets are regenerated at runtime from a stabilized RO-PUF response rather than stored in non-volatile memory. Under the stated protocol assumptions, passive storage disclosure therefore reveals no persistent private signing key, extracted master secret, root seed, or interval secret. Signing material exists only in volatile memory during on-demand authentication and is erased after use.
- 2) **A Merkle-root authorization mechanism bound to the PUF-derived identity.** During enrollment or periodic renewal, the UAV generates PUF-derived Merkle roots, the TA signs the roots, and authorized verifiers receive only the signed root records within their scope, without seeing the PUF-derived secrets, seeds, or future pseudonyms committed by the roots. This removes the need for TA connectivity before every flight and lets an authorized verifier check a disclosed Merkle leaf

against a PUF-derived, TA-signed commitment rather than a stored private credential.

- 3) **An explicit Remote ID standards and compliance boundary.** The protocol separates an experimental pseudonymous Specific Session ID profile from a supplementary-authentication profile that preserves the jurisdiction-required identifier and mandatory message set. The mapping distinguishes identifier semantics, message capacity, cadence, ASTM/DRIP authentication framing, transport integration, receiver interoperability, and regulatory certification rather than treating a 16-byte fit as proof of compliance.
- 4) **A bounded circuit–software co-simulation.** HSpice generates modeled 45 nm RO-PUF comparator outputs across process, voltage, temperature, and noise conditions, while a Raspberry Pi 5 executes response stabilization, fuzzy extraction, Merkle construction, and post-quantum authentication. Simulation instances are not treated as fabricated devices, and the evaluation makes no claim concerning silicon aging, invasive resistance, layout-dependent coupling, or physical side channels.

The resulting protocol provides compact pseudonymous broadcast, verifier-restricted Merkle disclosure, ML-KEM session establishment, and one transcript-bound ML-DSA authentication path. Section III defines the construction. Section IV examines broadcast linkability, impersonation, replay, verifier compromise, passive nonvolatile-memory disclosure, revocation, and the limits of active captured-device security. Section V separates broadcast, pre-flight, and on-demand authentication costs.

II. RELATED WORK

A. REMOTE ID STANDARDS, DRIP, AND COMPLIANCE BOUNDARY

Remote ID requirements extend beyond payload capacity. Under 14 CFR §89.305, a standard Remote ID aircraft broadcasts either a manufacturer-assigned serial number or a Session ID together with control-station location, aircraft position and altitude, velocity, time mark, and emergency status; a Remote ID broadcast module instead broadcasts its assigned module serial number under §89.315 [4]. The associated performance rules include synchronized time marks, self-testing, tamper resistance, non-proprietary broadcast, position and latency requirements, and transmission of the required message elements at least once per second [4]. ASTM F3411 defines 25-byte application messages and the Bluetooth/Wi-Fi transport mapping, while F3586 provides an FAA means-of-compliance practice [5], [6]. A standards mapping in an academic protocol is not equivalent to an FAA-accepted Means of Compliance or a manufacturer Declaration of Compliance [7].

The European legal basis has a corresponding product–operation split. Delegated Regulation (EU) 2019/945 specifies requirements for UAS products and direct-Remote-ID add-ons, whereas Implementing Regulation (EU) 2019/947 specifies operational rules and procedures [8], [9]. A deployment must

also satisfy the applicable harmonized technical standard and national implementation. Japan defines separate device and application requirements [10]. The cryptographic protocol does not replace these jurisdiction-specific obligations.

The IETF Drone Remote Identification Protocol (DRIP) work addresses trustworthy Remote ID through requirements and terminology, an architecture, DRIP Entity Tags (DETs), and ASTM Broadcast Authentication formats [23]–[26]. ASTM F3411-22a carries a DET through UAS ID Type 4, Specific Session ID: the first octet of the 20-octet UAS-ID field identifies the SSI subtype, leaving at most 19 octets for the subtype payload [24]. DRIP does not assign privacy to a DET; RFC 9374 states that DETs are broadcast in clear over Bluetooth and Wi-Fi and may persist for the life of a UA [24]. RFC 9575 uses ASTM Authentication Message Type 0x2, paged into 23-octet authentication payloads for legacy transports, and defines DRIP-specific authentication methods for public observers [26]. The present construction instead uses rotating pseudonyms and an ML-KEM/ML-DSA unicast proof restricted to authorized verifiers. It is therefore not a DRIP DET or DRIP Broadcast Authentication implementation.

A 128-bit pseudonym can fit within the 19-octet SSI subtype payload, but that observation establishes only message-size compatibility. The pseudonymous profile requires an SSI subtype and Session-ID semantics accepted by the applicable standards body or aviation authority. The paper assigns no SSI subtype value. The compliance-preserving profile leaves the approved Remote ID identity and complete mandatory message set unchanged and treats the PUF-rooted exchange as supplementary authentication. Table 1 states the resulting claim boundary.

The open broadcast still creates a privacy problem: any receiver in range can passively collect UAV activity, and prior surveys document identity exposure, location tracking, and targeted attacks against drones or pilots [1], [2]. Brighente et al. use geo-indistinguishability to obscure UAV location while preserving utility for authorized monitoring [27]; OLO-RID extends this to obfuscated disclosure that authorized entities can still resolve accurately [28]. Neither provides identity anonymity or cryptographic proof of legitimacy, and infrastructure-assisted alternatives such as UWB proof-of-location or blockchain validation solve a related but different problem: trust in a location claim, not compact authenticated identification [29].

B. PRIVACY-PRESERVING REMOTE ID PROTOCOLS

ARID, A2RID, and pqRID represent successive attempts at this problem. ARID resolves ephemeral pseudonyms only through a trusted authority, limiting independent local verification and relying on elliptic-curve cryptography with no quantum resistance [17]. A2RID removes the continuous trusted-third-party dependency using anonymous message-signing, at the cost of message-size and MAC-layer compatibility problems with standardized Remote ID modules [18]. pqRID, the closest prior work, decouples compact pseudonym

TABLE 1. Remote ID standards mapping and claim boundary.

Requirement	Governing source	Protocol treatment	Claim boundary
Permitted aircraft identity	14 CFR §89.305; ASTM Basic ID	Profile P uses a rotating PID only through an accepted Session-ID/SSI profile. Profile S retains the approved serial number or Session ID.	A self-defined PID is not asserted to be an accepted FAA identifier.
Broadcast-module identity	14 CFR §89.315	Profile S retains the manufacturer-assigned module serial number.	The rotating PID is not substituted for a broadcast-module serial number.
Mandatory message elements and cadence	14 CFR §§89.305–89.320; jurisdictional rules	The standards-conforming RID stack remains responsible for the complete message set, timing, accuracy, self-test, and at-least-1-Hz transmission.	The cryptographic protocol neither removes nor replaces mandatory messages.
ASTM identifier capacity	ASTM F3411-22a; RFC 9374	Profile P maps $SSIType \parallel PID_{k,j} \parallel 0^{24}$ into the 20-octet UAS-ID field, where $SSIType$ is externally assigned or accepted.	The 16-byte PID fits the field; size compatibility alone is not conformance.
Broadcast authentication	ASTM Authentication Message; RFC 9575	Merkle paths and ML-DSA signatures are delivered through authenticated unicast, not ASTM Type 0x2 pages.	The protocol provides authorized-verifier authentication, not public DRIP broadcast authentication.
Bluetooth/Wi-Fi framing	ASTM F3411 transport profiles	A standards-conforming transmitter supplies the BLE or Wi-Fi framing for the mandatory RID stack.	Link-layer conformance and packet-level interoperability are deployment evidence, not cryptographic properties.
FAA acceptance	FAA MOC and DOC procedures	No MOC or DOC is supplied by the protocol analysis.	FAA regulatory certification is not claimed.
EU product and operating rules	Regulations (EU) 2019/945 and 2019/947	Product integration and operation must satisfy both instruments and applicable harmonized standards.	EU conformity assessment and market approval are not claimed.
Independent receiver interoperability	ASTM/OpenDroneID-compatible receiver	A deployment evaluation must demonstrate BLE/Wi-Fi decoding, cadence, and complete message-set handling with independent receivers.	The host-side cryptographic prototype does not establish receiver interoperability.

broadcast from post-quantum authentication performed only on verifier demand, and was implemented on a Raspberry Pi 4B [19]. Its remaining gap is physical: pseudonym certificates and cryptographic material sit on the drone in stored form, exposed to extraction after capture. pqRID solves the broadcast-size problem, not PUF-derived credential regeneration under passive storage disclosure; that is where this paper's protocol contribution begins.

C. PUF-BASED UAV AND INTERNET-OF-DRONES AUTHENTICATION

PUFs derive cryptographic material from manufacturing-variation responses that are difficult to clone, avoiding permanent stored secret keys; fuzzy extractors turn noisy raw responses into stable secrets via public helper data [30], [31], [32]. Several UAV and Internet-of-Drones schemes build on this: a PUF-based hardware/software framework [20], a two-stage unlinkability-focused protocol [21], a formally validated drone-gateway protocol [22], and cross-domain, blockchain-supported PUF trust [33]. None targets Remote ID directly: their communication models assume mutual authentication between known parties, not continuous plaintext broadcast to arbitrary receivers, and none integrates Merkle-tree commitments with post-quantum verification. PUF security is also not unconditional; it depends on response entropy, modeling-attack resistance, environmental reliability, and restricting challenge-response exposure [34], which is why this framework exposes only stabilized, domain-separated secrets as KDF inputs rather than raw challenge-response pairs.

D. POST-QUANTUM UAV AUTHENTICATION

UAVs operate over open channels for years, making classical schemes a long-term liability against quantum-capable

adversaries [11]; NIST's ML-KEM, ML-DSA, and SLH-DSA standards give quantum-resistant alternatives [14]–[16]. The obstacle is size, not availability: ML-DSA signatures and public keys do not fit in Remote ID's short broadcast payloads [15], forcing the same architectural split between identification and proof that pqRID demonstrates [19]. Other work combines post-quantum mechanisms with PUFs for cross-domain security [33], but targets networked communication rather than regulatory broadcast. This framework moves post-quantum operations to an on-demand unicast phase while keeping compact PUF-derived pseudonyms in the broadcast phase.

E. MERKLE-TREE AND HASH-BASED VERIFICATION MECHANISMS

Merkle trees commit to a set of values with logarithmic-size membership proofs [35], already exploited in digital signatures, certificate transparency, and blockchain ledgers. For Remote ID, this separates routine broadcast from proof transmission: a UAV broadcasts only an interval identifier, and the Merkle proof travels later, only to an authorized verifier, so passive observers see a time-varying identifier with no way to link it to a long-term identity. Hash chains are a simpler alternative for time-evolving identifiers, but verifying one requires sequential state or reconstructing prior values. Merkle trees verify each interval independently, which tolerates intermittent requests, packet loss, and a verifier that starts observing mid-flight, exactly the conditions Remote ID traffic has. This framework pairs Merkle trees with PUF-derived interval identifiers so pseudonyms are both compact and hardware-bound.

F. LIGHTWEIGHT CRYPTOGRAPHY FOR UAV PLATFORMS

UAV authentication must balance security strength against latency, energy, memory, and code size; post-quantum public-key operations are too expensive to apply to every transmitted message when a symmetric primitive would do. Ascon, NIST's standardized choice for lightweight authenticated encryption, hashing, and extendable-output functions on constrained devices [36], [37], fills that role: once ML-KEM establishes an ephemeral shared secret, Ascon-based AEAD protects Merkle proofs and signature metadata during the authenticated unicast exchange.

G. COMPARATIVE ANALYSIS AND RESEARCH GAP

Table 2 places these schemes side by side. None satisfies the combined requirement this paper targets: compact plaintext broadcast, unlinkable pseudonymity, authorized traceability, post-quantum authentication, and resistance to extraction of stored drone secrets, all at once. The gap is not the absence of privacy, PUFs, Merkle trees, or post-quantum cryptography individually, since every one of those exists in prior work; it is their combined integration under Remote ID's specific operational constraints that remains unsolved.

The proposed framework addresses that gap by integrating RO-PUF-based secret regeneration, fuzzy extractors, Merkle-tree interval identifiers, ML-DSA signatures, ML-KEM key establishment, and Ascon-based authenticated encryption into a Remote-ID-constrained architecture with an explicit standards and certification boundary.

III. PROPOSED FRAMEWORK AND PROTOCOL

This section presents the PUF-rooted post-quantum authentication protocol for UAV Remote Identification. The protocol combines RO-PUF-based secret regeneration, fuzzy extraction, Merkle-root authorization, interval pseudonyms, and post-quantum authentication. A Merkle tree commits to the interval pseudonyms. In the pseudonymous Specific Session ID profile, the UAV broadcasts the active pseudonym through an externally accepted SSI mapping. In the supplementary-authentication profile, the jurisdiction-approved Remote ID identity remains in the broadcast and the active Merkle leaf is disclosed only through the protected verifier exchange.

The protocol runs in four phases.

- 1) **Phase 1: System initialization.**
- 2) **Phase 2: Enrollment and renewal.**
- 3) **Phase 3: Remote ID broadcast.**
- 4) **Phase 4: On-demand in-flight authentication.**

Each phase below is given as an algorithm listing that fixes the operation sequence, followed by prose that explains the design choices behind it. Merkle proofs, root authorization records, ML-KEM ciphertexts, and ML-DSA signatures are transmitted only during protected unicast authentication, never in the routine broadcast channel. The jurisdiction-required Remote ID message set is handled by the standards-conforming broadcast stack and is not reduced by the cryptographic protocol. The UAV keeps no persistent private signing key and

no permanent table of Merkle leaves or authentication paths in nonvolatile memory. It stores controlled PUF challenges, fuzzy-extractor helper data, public parameters, and compact TA-authorized root records instead, and it reconstructs the active Merkle paths locally, in volatile memory, before each flight. The four phases separate enrollment-time authorization, routine broadcast, and on-demand proof.

A. SYSTEM MODEL

Three entities make up the system: the UAV, the Trusted Authority, and the authorized verifier.

1) UAV

The protocol model assumes that UAV D_i carries an RO-PUF, a Remote ID broadcast interface, and an embedded processor capable of hash computation, fuzzy extraction, Merkle-tree construction, authenticated encryption, and post-quantum operations when required. It holds no long-term private signing key on disk. Signing material is regenerated from the stabilized PUF response in volatile memory only during an authorized authentication request and is erased immediately after use. The modeled RO-PUF is a 45 nm CMOS ring-oscillator array with 128 oscillators arranged in two banks of 64. The evaluated prototype does not contain a fabricated RO-PUF; it represents the response-vector interface with HSpice-generated comparator data and executes the remaining protocol stages on a Raspberry Pi 5. The co-simulation boundary is specified in Section V-C, and the response-vector analysis is specified in Section V-E.

2) Trusted Authority

The Trusted Authority, TA , handles UAV enrollment, verifier certification, Merkle-root authorization, renewal, revocation, and lawful traceability. It keeps two logically separated databases. The legal registration database holds the real UAV and operator identity; the operational authorization database holds public verification material, signed Merkle-root records, root validity intervals, authorization scopes, and verifier permissions. Operational verifiers see none of the operator identity unless a lawful traceability procedure specifically authorizes disclosure.

3) Authorized Verifier

An authorized verifier, V_j , might be an airport security receiver, a law-enforcement receiver, a critical-infrastructure monitoring unit, a ground control station, or another approved Remote ID verification entity. It can receive ordinary broadcasts and can initiate an authentication exchange when it needs proof of legitimacy. Verifiers receive signed Merkle-root records within their scope and nothing else: no PUF-derived secrets, root seeds, interval secrets, future pseudonyms, UAV private keys, or operator identity.

A passive observer sees only the broadcast. Under Profile P it observes changing pseudonymous identifiers; under Profile S it observes the jurisdiction-approved Remote ID identity and receives no cryptographic proof.

TABLE 2. Comparison of representative UAV Remote ID and authentication schemes.

Scheme / Study	Main objective	Remote ID standards treatment	Privacy / unlinkability	Local on-demand authentication	Post-quantum security
ARID [17]	Anonymous Remote ID using authority-resolvable pseudonyms	RID-oriented pseudonym broadcast; regulatory certification not established here	Yes	Limited; relies on trusted-authority resolution	No
A2RID [18]	Anonymous direct authentication and Remote ID	RID-oriented design constrained by message size and framing	Yes	Yes	No
pqRID [19]	Compact anonymous Remote ID with post-quantum authentication	Compact RID-oriented broadcast; identifier and framing conformance remain deployment-specific	Yes	Yes	Yes
PUF drone framework [20]	Hardware/software security for drones	No	Not Remote ID focused	Yes	No
TS-PAID [21]	Two-stage PUF-based IoD authentication	No	Yes	Yes	No
Nair et al. [33]	Post-quantum PUF-based cross-domain authentication for IoD	No	Cross-domain untraceability claimed; not a Remote ID pseudonym scheme	Yes	Yes

B. DEPLOYMENT PROFILES

The cryptographic core supports two deployment profiles. A profile-specific authentication reference *AuthRef* binds the verifier request to the identifier observed in the Remote ID broadcast:

$$AuthRef = \begin{cases} PID_{k,j}, & \text{Profile P,} \\ RIDID_i^{app}, & \text{Profile S.} \end{cases} \quad (1)$$

- **Profile P—pseudonymous Specific Session ID.** The ASTM Basic ID UAS-ID type is Specific Session ID. The 20-octet UAS-ID field is encoded as

$$UASID_{k,j}^P = SSIType \parallel PID_{k,j} \parallel 0^{24}, \quad (2)$$

where *SSIType* is a one-octet subtype assigned or accepted by the applicable standards body or aviation authority, *PID_{k,j}* occupies 16 octets, and three null octets complete the field. No numeric SSI subtype is allocated by this work. The verifier submits the observed pseudonym as *AuthRef*, and the UAV resolves the corresponding interval by scanning *ActiveState_k*. Its use depends on accepted Session-ID semantics and an applicable means of compliance.

- **Profile S—supplementary authorized-verifier authentication.** The standards-conforming RID transmitter broadcasts the jurisdiction-approved identity and all mandatory messages without modification. The verifier submits the observed approved identifier *RIDID_i^{app}* as *AuthRef*. The UAV verifies that it matches the active transmitter identity, selects the current interval pseudonym, and binds the approved identifier, current pseudonym, and remaining transcript into the ML-DSA signature. Profile S preserves the regulatory broadcast but does not provide identifier-level broadcast pseudonymity.

Neither profile changes mandatory message cadence, accuracy, self-test, error-correction, transport, or operator-identification requirements. Profile P establishes a packet mapping that remains conditional on external acceptance. Profile S can be integrated without replacing the approved identifier, but its post-quantum proof is available only to authorized verifiers and is not an ASTM/DRIP Broadcast Authentication Message.

C. NOTATION AND CRYPTOGRAPHIC PRIMITIVES

Table 3 lists the notation used throughout this section.

Three index conventions recur across every equation and

TABLE 3. Notation used in the proposed protocol.

Symbol	Description	Symbol	Description
D_i	The i -th UAV	V_j	The j -th authorized verifier
TA	Trusted Authority	ID_i	Legal UAV identity (TA database only)
VID_j	Identity of verifier V_j	$CSeed$	Public seed defining the ordered RO-PUF challenge schedule
$\mathcal{C}$	Ordered schedule of $q = 1023$ distinct 12-bit RO-pair challenges	R, R'	Enrollment and reconstruction response vectors in $\{0, 1\}^{1023}$
$FE.Gen(\cdot)$	Fuzzy-extractor enrollment function	$FE.Rec(\cdot)$	Fuzzy-extractor reconstruction function
$HD = (s, \rho)$	Public BCH syndrome and extractor seed	K_{PUF}	Reconstructed 256-bit PUF master secret
S_1, S_2	Domain-separated signing and Merkle secrets	$H_{256}(\cdot), H_{384}(\cdot)$	256-bit general-purpose hash and 384-bit collision-sensitive tuple hash
$KDF(\cdot)$	Key-derivation function	$RootNonce_k$	Root-specific nonce for window k
$Seed_k$	PUF-derived root seed	$X_{k,j}$	Interval secret for interval j under root k
$PID_{k,j}$	Pseudonymous identifier for interval j	$L_{k,j}$	Merkle leaf corresponding to $PID_{k,j}$
$MT_i^{(k)}$	Merkle tree of UAV D_i under root k	$MR_i^{(k)}$	Root of $MT_i^{(k)}$
$\pi_{k,j}$	Authentication path for $L_{k,j}$	$AuthRec_i^{(k)}$	TA-signed authorization record
$FlightCtx_k$	Authorization context for root k	$ActiveState_k^{(k)}$	Volatile pseudonym and tree state
KC_i	Key-confirmation hash $H_{256}(PK_i)$	$j_{next}^{(k)}$	Lowest interval index not yet durably reserved
g_k	Journal generation number	$J_{k,0}, J_{k,1}$	Alternating nonvolatile journal slots
chk_k	Journal-record checksum	$ReserveNext$	Durable reservation of $(k, j + 1)$ before first use
PK_i, SK_i	UAV ML-DSA key pair	PK_{TA}, SK_{TA}	TA ML-DSA key pair
PK_{V_j}, SK_{V_j}	Verifier ML-DSA key pair	$Cert_{V_j}$	TA-issued verifier certificate
RL_v	Versioned TA revocation list	σ_{RL_v}	TA signature on $H_{256}(Enc(RL_v))$
v_{last}	Highest retained revocation-list version	ek_{V_j}, dk_{V_j}	Ephemeral verifier ML-KEM key pair
ct	ML-KEM ciphertext	ss	ML-KEM shared secret
K_{sess}	Session key derived from ss	N_V	Verifier nonce
$RIDSet_j$	Jurisdiction-required Remote ID message set at interval j	$RIDID_i^{app}$	Jurisdiction-approved Remote ID identity
$SSIType$	Externally assigned or accepted Specific Session ID sub-type	$AuthRef$	Profile-specific identifier reference bound to the observed broadcast
j_{cur}	Active interval index during a verifier request	TS	Timestamp

algorithm that follows, and keeping them straight is the main reading aid for the rest of this section: i indexes a UAV, (k) indexes an authorized Merkle-root window belonging to that UAV, and j indexes an interval within that window. A term such as $X_{k,j}$ is the interval secret for interval j under root k ; $AuthRec_i^{(k)}$ is the TA's authorization record for UAV D_i 's root k . Once these three roles are separated in the reader's mind, the compound subscripts stop being an obstacle.

The primitives are standard: an RO-PUF for hardware-derived responses, a syndrome-based fuzzy extractor for response stabilization, H_{256} for pseudonym generation, request hashing, key confirmation, revocation records, and journal checksums, and H_{384} for collision-sensitive Merkle and transcript hashing. In the implementation, H_{256} is Ascon-Hash256 and H_{384} is TupleHash256 with a 384-bit output as specified in NIST SP 800-185 [38]. TupleHash supplies unambiguous, length-delimited encoding of structured inputs. SHAKE256 derives purpose-specific 256-bit secrets; a Merkle tree commits to planned pseudonyms; ML-DSA provides post-quantum signatures; ML-KEM establishes post-quantum sessions; and AEAD protects proof material once a session key exists.

The RO-PUF interface returns one comparator bit for each challenge. It does not treat the challenge width as the response width. A public seed $CSeed$ defines an ordered schedule $\mathcal{C} = \{c_1, \dots, c_q\}$ of $q = 1023$ distinct 12-bit challenges. Each

challenge selects one oscillator from each of two 64-oscillator banks. Nine comparator readings are taken for each challenge and reduced by majority vote:

$$R_\ell = \text{Maj}(PUF(c_\ell)^{(1)}, \dots, PUF(c_\ell)^{(9)}), \quad \ell = 1, \dots, q. \quad (3)$$

The resulting vector $R = (R_1, \dots, R_q) \in \{0, 1\}^{1023}$ is processed by a syndrome-based fuzzy extractor [32]. Let H_{BCH} be the parity-check matrix of a binary BCH(1023, 943, 17) code, which corrects up to $t = 8$ bit errors and has 80 parity bits. Enrollment computes

$$s = H_{BCH}R^T, \quad K_{PUF} = Ext_\rho(R), \quad HD = (s, \rho), \quad (4)$$

where Ext_ρ is a seeded 2-universal Toeplitz extractor with a 256-bit output and public seed ρ . During reconstruction, the UAV acquires a noisy vector R' , computes the syndrome difference, decodes an error vector e of Hamming weight at most eight, and recovers

$$\hat{R} = R' \oplus e, \quad K_{PUF} = Ext_\rho(\hat{R}). \quad (5)$$

Decoder failure causes the protocol operation to abort; an uncorrected vector is never passed to key generation. Two purpose-specific secrets are derived from the same stabilized master secret:

$$\begin{aligned} S_1 &= \text{SHAKE256}_{256}(\text{"PUF-SIGN"} \parallel K_{\text{PUF}} \parallel \text{ctx}_i), \quad (6) \\ S_2 &= \text{SHAKE256}_{256}(\text{"PUF-MERKLE"} \parallel K_{\text{PUF}} \parallel \text{ctx}_i). \quad (7) \end{aligned}$$

The extractor output length alone does not establish its security strength. The response-vector characterization must provide a lower confidence bound $H_{\infty}^{LB}(R) \geq 464$ bits. Since the BCH syndrome reveals at most 80 bits, this threshold leaves at least 384 bits of conditional min-entropy. A 256-bit 2-universal extraction then has statistical distance at most 2^{-64} from uniform under the leftover-hash bound [32]. This is an explicit 64-bit statistical-distance target and is not presented as NIST category-3 security. Achieving a target of 2^{-192} with the same 256-bit output would require at least 640 bits of conditional min-entropy, or approximately 720 bits before the 80-bit syndrome leakage; the present construction does not claim that stronger bound without corresponding measurements. The resulting S_1 supplies the 32-byte seed required by deterministic ML-DSA key expansion under FIPS 204 [15].

D. CANONICAL WIRE FORMAT AND PROTOCOL STATE

All signed, hashed, KDF, and AEAD inputs use a single canonical binary encoding. The encoder $\text{Enc}(\cdot)$ emits a fixed eight-octet envelope followed by strictly ordered length-delimited fields:

$$\text{Wire} = \text{Magic}_{16} \parallel \text{Ver}_8 \parallel \text{MsgType}_8 \parallel \text{PayloadLen}_{32} \parallel \text{Fields}, \quad (8)$$

where $\text{Magic} = 0x5249$ ("RI"), $\text{Ver} = 0x01$, and PayloadLen is the number of octets following the envelope. Every field is encoded as

$$\text{Field} = \text{Type}_{16} \parallel \text{Len}_{32} \parallel \text{Value}, \quad (9)$$

with unsigned integers in network byte order, fixed-width integer values, UTF-8 prohibited inside cryptographic objects, and fields sorted by ascending Type . Duplicate fields, non-minimal encodings, unknown critical field types, inconsistent lengths, trailing octets, and noncanonical order are rejected before signature or hash verification. Variable-length objects such as certificates, authorization records, and scopes are therefore unambiguous. The signed object is the exact canonical wire image rather than an implementation-language object or JSON representation.

The profile-specific request identifier is

$$\text{ReqID} = \text{Trunc}_{128}(H_{256}(\text{"ReqID"} \parallel \text{ReqAuthWire})). \quad (10)$$

The verifier signs $H_{256}(\text{ReqAuthWire})$, and every subsequent transcript value binds the same byte string. ML-KEM produces a fresh session secret for each accepted request. Ascon-AEAD128 uses a 128-bit key K_{sess} and the deterministic per-session nonce

$$N_A = \text{Trunc}_{128}(H_{256}(\text{"AEAD-Nonce"} \parallel \text{ReqID} \parallel \text{ct})). \quad (11)$$

Because ct is generated by a fresh ML-KEM encapsulation and ReqID binds the verifier nonce, nonce reuse under the same K_{sess} is excluded for accepted sessions. The associated data are

$$AD_A = \text{Enc}(\text{RespHeader}, \text{ReqID}, \text{ct}, N_A, H_{256}(\text{ReqAuthWire})), \quad (12)$$

so the unencrypted response header, KEM ciphertext, request identity, and encrypted payload are cryptographically bound. The response packet order is RespOuter ; the verifier first decapsulates ct , derives K_{sess} , and then verifies the AEAD tag before parsing PayloadWire . No field from unauthenticated plaintext is acted upon before AEAD verification.

Merkle construction uses a complete binary tree with $n = 2^d$ leaves. A leaf is

$$L_{k,j} = H_{384}(0x00, \text{Enc}(\text{PID}_{k,j}, \text{RootNonce}_k, \text{FlightCtx}_k)), \quad (13)$$

and an internal node at level ℓ is

$$\text{Node}_{\ell,r} = H_{384}(0x01, \text{Enc}(\ell, \text{Node}_{\ell-1,2r}, \text{Node}_{\ell-1,2r+1})). \quad (14)$$

The authentication path is encoded from leaf level to root level as d 48-octet sibling values followed by a d -bit direction vector, where zero denotes a left sibling and one a right sibling. A verifier rejects a path with the wrong depth, wrong direction-vector length, extra nodes, or a root mismatch. The distinct leaf and internal-node tags prevent cross-type substitution.

Replay resistance does not depend on timestamps alone. Each UAV maintains ReplayCache entries of the form $(\text{CertID}_j, N_V, \text{ReqID}, \text{Expiry})$ for every accepted verifier request. A request is rejected when its tuple is already present. Entries are retained until at least $TS_V + \Delta_{\text{fresh}} + T_{\text{session}}^{\text{max}}$, and the cache is sized for the maximum authenticated request rate over that interval. A cache-capacity failure causes fail-closed rejection rather than eviction of a still-valid entry. N_V is sampled uniformly from 128 bits.

The UAV uses a rollback-protected monotonic clock for local ordering and a UTC source for policy times. A request passes freshness only when $|TS_{\text{local}} - TS_V| \leq \Delta_{\text{fresh}}$, the local monotonic time has not moved backward, and the active root and revocation list are valid at TS_{local} . Loss of trustworthy time causes authentication to fail closed; the broadcast stack may continue only according to the applicable Remote ID rules. TS_D is generated after verifier acceptance and is checked by the verifier against the same configured window.

Scope authorization is evaluated as a set-intersection predicate rather than by string equality:

$$\text{ScopeOK} \equiv \text{RequestedScope} \subseteq \text{Scope}(\text{Cert}_V) \cap \text{Scope}_k \cap \text{Scope}_{\text{policy}}(R) \quad (15)$$

The request is rejected if any requested role, region, operation class, time window, or data-disclosure permission lies outside that intersection.

Three stable identifiers support revocation without exposing legal identity:

$$\text{CertID}_j = H_{256}(\text{Cert}_V), \quad \text{RootID}_k = H_{256}(\text{AuthRec}_i^{(k)}), \quad \text{KeyID}_i = \quad (16)$$

TABLE 4. Canonical messages used by the on-demand authentication exchange.

Message	MsgType	Canonical field order
<i>ReqAuthWire</i>	0x01	$Cert_{V_j}$, ek_{V_j} , $AuthRef$, N_V (16 octets), TS_V (unsigned 64-bit seconds), $RequestedScope$, v (unsigned 64-bit), $RIDSetDigest$ (32 octets; Profile S, zero length in Profile P).
<i>RespOuter</i>	0x02	$ReqID$ (16 octets), ct (1088 octets for ML-KEM-768), N_A (16 octets), $Ciphertext$, Tag (16 octets).
<i>PayloadWire</i>	0x03	$AuthRef$, $PID_{k,j_{cur}}$ (16 octets), j_{cur} (unsigned 32-bit), $RootNonce_k$ (32 octets), $FlightCtx_k$, ordered Merkle path, $MR_i^{(k)}$ (48 octets), $AuthRec_i^{(k)}$, TS_D (unsigned 64-bit), σ_D (3309 octets).

The revocation list can therefore invalidate a verifier certificate, one authorization root, or the UAV signing identity across all roots. UAV-key revocation is mandatory after confirmed active-device capture or signing-key compromise.

E. PHASE 1: SYSTEM INITIALIZATION

System initialization runs once, at the TA. It fixes the public protocol parameters:

$$Params = \{H, KDF, SHAKE256, FE, ML-DSA, ML-KEM, AEAD, \Delta T, n, m, \lambda\}. \quad (17)$$

where ΔT is the Remote ID pseudonym interval, n is the number of interval identifiers committed under one Merkle root, m is the number of roots issued per enrollment or renewal cycle, and λ is the security parameter. The TA then generates its own post-quantum signing key pair,

$$(PK_{TA}, SK_{TA}) \leftarrow ML-DSA.KeyGen(), \quad (18)$$

and distributes PK_{TA} to enrolled UAVs and authorized verifiers. Every verifier certificate and every Merkle-root authorization record traces back to this one key.

Parameter selection. *Params* fixes ML-DSA-65 for every TA, verifier, and UAV signature and ML-KEM-768 for session establishment. FIPS 203 and FIPS 204 assign both primitives to NIST security category 3, but those primitive classifications do not establish a uniform category-3 level for the complete protocol. System-level guarantees remain property-dependent and are also bounded by the hash output used for collision-sensitive commitments, the 128-bit pseudonym space, the fuzzy-extractor statistical distance, the deployment-specific conditional PUF entropy assumption, and the implementation assumptions. At this parameter set, an ML-DSA-65 public key is 1952 bytes and a signature is 3309 bytes, while an ML-KEM-768 public key is 1184 bytes and its ciphertext is 1088 bytes [14], [15]. These objects remain outside the broadcast channel and are carried only in the unicast exchanges of Phase 4. For category-5 deployments, ML-DSA-87 uses a 2592-byte public key and a 4627-byte signature, and ML-KEM-1024 uses a 1568-byte public key and a 1568-byte ciphertext. Relative to the selected category-3 parameters, these increases are approximately 32.8%, 39.8%, 32.4%, and 44.1%, respectively.

Domain separation and encoding. Every KDF , H_{256} , and H_{384} call is tagged with a fixed, distinct label listed in Table 5.

TABLE 5. Domain-separation labels used in KDF and hash calls.

Label	Used in
"PUF-SIGN"	Signing-secret derivation from K_{PUF}
"PUF-MERKLE"	Merkle-secret derivation from K_{PUF}
"MerkleRoot"	Root seed derivation
"IntervalSecret"	Interval secret derivation
"RID"	Pseudonym generation
"leaf"	384-bit Merkle leaf hashing
"node"	384-bit ordered Merkle internal-node hashing
"RID-Auth"	Session-key derivation
"RID-UAV-Auth-v1"	384-bit authentication-transcript hashing
"state"	Interval-journal record checksum

Structured H_{384} inputs are encoded as TupleHash tuples, which preserve field boundaries and order. For readability, the equations retain the symbol $\|$; the implementation interprets it as canonical, length-delimited serialization with fixed field types, explicit integer widths, network byte order, and a protocol-version field rather than raw ambiguous concatenation. No two derivations share a label, and Merkle internal nodes include an explicit node label and ordered left/right children.

F. PHASE 2: ENROLLMENT AND RENEWAL

Enrollment and renewal happen before operational flight, over a secure channel, and remove any need for the UAV to reach the TA during a mission. The phase covers three procedures: the UAV enrolls and receives Merkle-root authorization, a verifier enrolls separately and receives a scoped slice of the TA's authorization database, and either side can trigger renewal once a root is exhausted or its validity window closes.

1) UAV Enrollment and Merkle-Root Authorization

The UAV operator first registers with the TA out of band, submitting operator identity, UAV serial number, manufacturer, model, and whatever regulatory data applies. The TA checks this and assigns a legal identity ID_i , which stays in the legal registration database and is never handed to operational verifiers by default.

What follows on the UAV side is a fixed sequence: acquire one majority-voted RO-PUF response vector, reconstruct a 256-bit master secret, derive separate signing and Merkle

Algorithm 1 UAV Enrollment and Merkle-Root Authorization

Require: Public schedule seed $CSeed$; context string ctx ; window count m ; interval count n

Ensure: Signing key pair (PK_i, SK_i) ; authorization records $\{AuthRec_i^{(k)}\}_{k=1}^m$

- 1: $C \leftarrow ChallengeSchedule(CSeed, q = 1023)$
- 2: $R \leftarrow MajorityPUF(C, 9)$
- 3: $(K_{PUF}, HD) \leftarrow FE.Gen(R)$
- 4: $S_1 \leftarrow SHAKE256_{256}("PUF-SIGN" \parallel K_{PUF} \parallel ctx_i)$
- 5: $S_2 \leftarrow SHAKE256_{256}("PUF-MERKLE" \parallel K_{PUF} \parallel ctx_i)$
- 6: $(PK_i, SK_i) \leftarrow ML-DSA.KeyGen_{internal}(S_1)$
- 7: $KC_i \leftarrow H_{256}(PK_i)$
- 8: Register PK_i with the TA; discard SK_i from persistent storage
- 9: **for** $k = 1$ **to** m **do**
- 10: **Build Merkle-root window k from S_2 .**
- 11: $FlightCtx_k \leftarrow \{k, \Delta T, n, Validity_k, Scope_k, Region_k\}$
- 12: **repeat**
- 13: $RootNonce_k \xleftarrow{\$} \{0, 1\}^\lambda$
- 14: $Seed_k \leftarrow KDF(S_2 \parallel RootNonce_k \parallel k \parallel "MerkleRoot")$
- 15: **for** $j = 1$ **to** n **do**
- 16: $X_{k,j} \leftarrow KDF(Seed_k \parallel j \parallel "IntervalSecret")$
- 17: $PID_{k,j} \leftarrow Trunc_{128}(H_{256}("RID" \parallel X_{k,j} \parallel j \parallel RootNonce_k \parallel FlightCtx_k))$
- 18: $L_{k,j} \leftarrow H_{384}(0x00, Enc(PID_{k,j}, j, RootNonce_k, FlightCtx_k))$
- 19: **end for**
- 20: **if** $|\{PID_{k,1}, \dots, PID_{k,n}\}| < n$ **then**
- 21: Erase $Seed_k$ and all interval values for window k
- 22: **end if**
- 23: **until** $|\{PID_{k,1}, \dots, PID_{k,n}\}| = n$
- 24: $MT_i^{(k)} \leftarrow MerkleTree_{Enc, H_{384}}(L_{k,1}, \dots, L_{k,n})$
- 25: $MR_i^{(k)} \leftarrow Root(MT_i^{(k)})$
- 26: Send $RootReq_i^{(k)}$ to the TA over the secure channel
- 27: Receive $AuthRec_i^{(k)}$
- 28: Initialize $J_{k,0} \leftarrow \{0, k, 1, H_{256}("state" \parallel 0 \parallel k \parallel 1)\}$ and $J_{k,1} \leftarrow \perp$
- 29: **end for**
- 30: Erase R, K_{PUF}, S_1, S_2 , every $Seed_k$, and every $X_{k,j}$ from volatile memory

secrets, and build m Merkle-root commitments. Algorithm 1 lays this out.

Merkle leaves and internal nodes use 384-bit TupleHash outputs. For an ordered pair of children at level ℓ and position r , the parent is

$$P_{\ell,r} = H_{384}("node", \ell, r, P_{\ell-1,2r}, P_{\ell-1,2r+1}), \quad (19)$$

where left/right order, level, and position are part of the tuple. The 128-bit PID remains an operational broadcast identifier rather than a category-3 authenticator. Assuming pseudorandom, independently distributed identifiers, the probability that a window of n values contains at least one collision is bounded by

$$\Pr[Coll_{PID}] \leq \frac{n(n-1)}{2^{129}}. \quad (20)$$

Algorithm 1 rejects any window containing a duplicate and regenerates it under a fresh $RootNonce_k$ before requesting TA authorization.

Two message formats are worth spelling out on their own, since a reader checking the protocol for correctness needs to see exactly what crosses the wire and what lands in storage. Sending its root request, the UAV transmits

$$RootReq_i^{(k)} = \{PK_i, MR_i^{(k)}, RootNonce_k, FlightCtx_k, Validity_k, Scope_k\}. \quad (21)$$

and the TA, once satisfied, replies with a signed authorization record,

$$AuthRec_i^{(k)} = Sign_{SK_{TA}}(PK_i \parallel MR_i^{(k)} \parallel RootNonce_k \parallel FlightCtx_k \parallel Validity_k \parallel Scope_k). \quad (22)$$

while recording the legal traceability mapping $MR_i^{(k)} \rightarrow ID_i$ inside the protected legal registration system, kept apart from anything a verifier can see.

On the UAV side, only a small, non-secret residue survives in nonvolatile memory:

$$Store_{D_i}^{NV} = \{CSeed, HD, ctx_i, Params_i, KC_i, RL_v, \sigma_{RL_v}, v_{last}, RootNonce_k, FlightCtx_k, AuthRec_i^{(k)}, k, Validity_k, J_{k,0}, J_{k,1}\}_{k=1}^m. \quad (23)$$

The stored helper data are public reconstruction data rather than a secret key. The raw response vector R , the corrected vector $\hat{R}$, K_{PUF} , S_1 , S_2 , $Seed_k$, $X_{k,j}$, and SK_i are absent from nonvolatile storage. The security of this arrangement depends on a deployment-specific lower bound on the conditional min-entropy of the response vector after the public syndrome has been revealed. The co-simulation reports a model-specific estimate, while a physical deployment must establish the corresponding bound on manufactured devices; helper-data exposure is included explicitly in both analyses. KC_i is a hash of the public PK_i and is used only to detect a failed deterministic key reconstruction before signing.

2) Verifier Enrollment and Merkle-Root Renewal

Verifier enrollment is close to the mirror image of UAV enrollment, minus the PUF: a verifier generates its own key pair, the TA certifies the public half against a declared scope, and the TA hands back only the slice of its authorization database that scope covers. Renewal reuses the Merkle-root construction from Algorithm 1 with fresh nonces, triggered whenever a UAV exhausts its intervals, a validity window closes, or policy calls for rotation. Algorithm 2 covers both procedures.

Nothing in $AuthDB_{V_i}$ lets a verifier compute a future pseudonym: it never receives S_1 , S_2 , $Seed_k$, $X_{k,j}$, $PID_{k,j}$, SK_i , or ID_i . Expired or consumed roots are marked invalid at the TA. UAVs and verifiers obtain revocation state through the periodically synchronized, TA-signed lists defined in Section III-I.

Table 6 lists what each stage of Phase 2 consumes and what it leaves behind.

Algorithm 2 Verifier Enrollment and Merkle-Root Renewal

```

1: Verifier enrollment.
2:  $V_j$  submits  $Req_{V_j} = \{VID_j, Org_j, Role_j, Scope_j, Validity_j\}$ 
3:  $V_j$  generates  $(PK_{V_j}, SK_{V_j}) \leftarrow ML\text{-}DSA.KeyGen()$  and keeps  $SK_{V_j}$  locally
4: TA issues  $Cert_{V_j} \leftarrow Sign_{SK_{TA}}(VID_j \parallel PK_{V_j} \parallel Scope_j \parallel Validity_j)$ 
5: TA forms the scoped database
6:  $AuthDB_{V_j} \leftarrow \{PK_i, MR_i^{(k)}, RootNonce_k, FlightCtx_k, Validity_k, Scope_k, AuthRec_i^{(k)}\}_{i,k \in Scope_j}$ 
7: Merkle-root renewal (triggered on expiry, exhaustion, or scope change).
8:  $R' \leftarrow MajorityPUF(ChallengeSchedule(CSeed, q = 1023), 9)$ 
9:  $K_{PUF} \leftarrow FE.Rec(R', HD)$ 
10:  $S_2 \leftarrow SHAKE256_{256}(\text{"PUF-MERKLE"} \parallel K_{PUF} \parallel ctx_i)$ 
12: UAV generates fresh  $RootNonce_{k'}$  and repeats the interval-derivation loop of Algorithm 1 to obtain  $MR_i^{(k')}$ 
13: UAV sends  $RenewReq_i^{(k')}$  containing
14:    $\{PK_i, MR_i^{(k')}, RootNonce_{k'}, FlightCtx_{k'}, Validity_{k'}, Scope_{k'}\}$ 
15: Erase  $R', K_{PUF}$ , and  $S_2$ 
16: TA verifies registration and validity, then issues  $AuthRec_i^{(k')}$  following Eq. (22)
17: Initialize  $J_{k',0} \leftarrow \{0, k', 1, H_{256}(\text{"state"} \parallel 0 \parallel k' \parallel 1)\}$  and  $J_{k',1} \leftarrow \perp$ 

```

TABLE 6. Phase 2 summary: inputs and outputs by stage.

Stage	Consumes	Produces / retains
Response acquisition and extraction	$CSeed$	K_{PUF} , HD ; master secret volatile only
Domain separation and signing key	K_{PUF}	$S_1, S_2, (PK_i, SK_i)$; secrets and SK_i discarded
Root construction ($\times m$)	S_2 , $RootNonce_k$	$MR_i^{(k)}$, $AuthRec_i^{(k)}$
Verifier enrollment	Req_{V_j}	$Cert_{V_j}$, $AuthDB_{V_j}$
Renewal	Fresh $RootNonce_{k'}$	$AuthRec_i^{(k')}$

G. PHASE 3: REMOTE ID BROADCAST

Broadcast is the routine phase. The UAV activates one authorized root locally without contacting the TA. Root activation reconstructs the Merkle tree in volatile memory, and interval use is controlled by a crash-consistent reservation record maintained in nonvolatile storage. At each rotation boundary, the standards-conforming RID stack emits the jurisdiction-required message set $RIDSet_j$. Under Profile P, its Basic ID field carries $UASID_{k,j}^P$ from Eq. (2); under Profile S, the Basic ID field retains $RIDID_i^{app}$.

For root k , the persistent interval state is stored in two alternating journal slots,

$$J_{k,b} = \{g_k, k, j_{next}^{(k)}, chk_k\}, \quad b \in \{0, 1\}, \quad (24)$$

$$chk_k = H_{256}(\text{"state"} \parallel g_k \parallel k \parallel j_{next}^{(k)}).$$

where g_k is a monotonically increasing generation number and $j_{next}^{(k)}$ is the lowest interval index not yet reserved. On recovery, the UAV selects the valid slot with the largest generation

Algorithm 3 Pre-Flight Root Activation

```

Require: Stored  $AuthRec_i^{(k)}$ ; recovered  $j_{next}^{(k)}$ ; valid cached  $RL_v$ 
1: if  $TS \notin [ThisUpdate_v, NextUpdate_v]$  or  $RootID_k \in RevokedRoots_v$  or  $KeyID_i \in RevokedUAVKeys_v$  then
2:   reject the root and obtain a current revocation list
3:   return
4: end if
5: if  $j_{next}^{(k)} > n$  then
6:   select another authorized record or trigger renewal
7:   return
8: end if
9:  $C \leftarrow ChallengeSchedule(CSeed, 1023)$ 
10:  $R' \leftarrow MajorityPUF(C, 9)$ 
11:  $K_{PUF} \leftarrow FE.Rec(R', HD)$ 
12:  $S_2 \leftarrow SHAKE256_{256}(\text{"PUF-MERKLE"} \parallel K_{PUF} \parallel ctx_i)$ 
13:  $Seed_k \leftarrow KDF(S_2 \parallel RootNonce_k \parallel k \parallel \text{"MerkleRoot"})$ 
14: Recompute  $X_{k,j}$ ,  $PID_{k,j}$ , and  $L_{k,j}$  for  $j = 1, \dots, n$ 
15:  $MT_i^{(k)} \leftarrow MerkleTree_{Enc, H_{384}}(L_{k,1}, \dots, L_{k,n})$ 
16: if  $Root(MT_i^{(k)}) \neq MR_i^{(k)}$  then
17:   reject the root and return
18: end if
19:  $ActiveState_k \leftarrow \{PID_{k,j}, L_{k,j}, \pi_{k,j}\}_{j=1}^n$ 
20: Erase  $R', K_{PUF}$ ,  $S_2$ ,  $Seed_k$ , and interval secrets

```

Algorithm 4 Crash-Consistent Pseudonym Reservation and Broadcast

```

Require: Active  $ActiveState_k$  and recovered  $j_{next}^{(k)}$ 
1: while flight active do
2:   Wait until the next pseudonym-rotation boundary
3:    $j \leftarrow j_{next}^{(k)}$ 
4:   if  $j > n$  then
5:     stop using root  $k$  and select another authorized root
6:     break
7:   end if
8:   if  $ReserveNext(k, j+1) = 0$  then
9:     suppress transmission and enter a fail-safe state
10:    break
11:   end if
12:   Emit the complete  $RIDSet_j$  through the standards-conforming RID stack
13:   In Profile P, set  $BasicID.UASID \leftarrow UASID_{k,j}^P$ ; in Profile S, retain  $RIDID_i^{app}$ 
14: end while

```

number; if neither slot validates, it fails closed and does not use that root. The operation $ReserveNext(k, j+1)$ writes the next generation to the inactive slot, forces the record to durable storage, verifies the stored checksum, updates the in-memory value of $j_{next}^{(k)}$, and returns success only after the new value can be recovered following a restart. The checksum detects incomplete accidental writes with negligible false-acceptance probability under the hash assumption; adversarial rollback or modification of the journal belongs to active device compromise. This reserve-before-use ordering follows the write-ahead persistence principle [39].

Under Profile P, a pseudonym may be repeated in ordinary Remote ID packets within its assigned rotation interval. It is not reused in a later rotation interval or after restart. The next index is durably reserved before the first packet containing $PID_{k,j}$ is transmitted. A crash after transmission therefore recovers $j_{next}^{(k)} \geq j+1$ and cannot return to j . If the crash occurs after the reservation commit but before the first transmission, index j is consumed without being broadcast. Under Profile S, the same

reservation rule assigns a unique internal interval pseudonym to the authentication state even though the approved broadcast identifier remains unchanged. The protocol consequently provides at-most-once interval-pseudonym use rather than gap-free consumption.

Skipped indices do not invalidate the Merkle commitment because verifiers check membership of the pseudonym that was actually observed. They reduce the remaining capacity of the active root. The number of skipped indices is bounded by the number of crashes occurring between durable reservation and first transmission; repeated failures may exhaust a root earlier and trigger local selection of another authorized record or renewal. Recovery requires no TA contact while another valid authorized root remains available.

The wall clock determines when the pseudonym-rotation boundary occurs, while $j_{next}^{(k)}$ determines which unused Merkle leaf is assigned to that boundary. $ActiveState_k$ remains volatile and is cleared after the flight, root expiration, or abnormal termination. The no-reuse property depends on the durable-commit semantics of $ReserveNext$; a buffered write that has not reached the persistence domain is not treated as a successful reservation.

The broadcast never contains $MR_i^{(k)}$, $\pi_{k,j}$, $RootNonce_k$, S_2 , $Seed_k$, $X_{k,j}$, or SK_i . Under Profile P, a passive receiver collects the transmitted pseudonym sequence without an attached proof. Under Profile S, it collects the approved Remote ID identifier and mandatory telemetry, while the interval pseudonym remains local until an authorized exchange. The Merkle root is a signed commitment, not a routine broadcast value. Confirming membership requires the authentication path and the protected exchange in Phase 4.

H. PHASE 4: ON-DEMAND IN-FLIGHT AUTHENTICATION

An authorized verifier that needs proof for an observed Remote ID transmission opens a unicast session with the UAV. Phase 4 has two consecutive steps. Algorithm 5 authenticates the verifier, validates the profile-specific $AuthRef$, and establishes an ML-KEM session key. Algorithm 6 then releases the relevant Merkle path and a PUF-regenerated ML-DSA signature through the protected channel. The protocol has no lower-assurance branch and discloses no interval secret.

1) Verifier Authentication and Session Establishment

Everything released after verifier acceptance is protected by AEAD under K_{sess} . In Profile P, the request carries the pseudonym exactly as observed on the broadcast channel and reveals neither the associated root nor interval index. The UAV resolves that index by scanning all n entries in $ActiveState_k$ without data-dependent early exit. In Profile S, the request carries the approved identifier and a canonical digest of the observed message set; the UAV verifies them against its current RID state and uses the active interval index. A stale, foreign, unmatched, or inconsistent reference receives the same silent failure as an invalid verifier request.

Algorithm 5 Verifier Authentication and Session Establishment

```

1:  $V_j$  generates nonce  $N_V$  and ephemeral pair  $(ek_{V_j}, dk_{V_j}) \leftarrow$ 
    $ML-KEM.KeyGen()$ 
2:  $V_j$  forms canonical  $ReqAuthWire \leftarrow$ 
    $Enc(Cert_{V_j}, ek_{V_j}, AuthRef, N_V, TS_V, RequestedScope, v, RIDSetDigest)$ 
   using Table 4
3:  $V_j$  computes  $ReqID$  from Eq. (10) and  $\sigma_{V_j} \leftarrow$ 
    $Sign_{SK_{V_j}}(H_{256}(ReqAuthWire))$ 
4:  $V_j$  sends  $\{ReqAuthWire, \sigma_{V_j}\}$  to  $D_i$ 
5:  $D_i$  canonically decodes  $ReqAuthWire$ , verifies  $Cert_{V_j}$  under  $PK_{TA}$ , and
   checks  $Verify_{PK_{V_j}}(H_{256}(ReqAuthWire), \sigma_{V_j}) = 1$ 
6:  $D_i$  verifies  $RL_V$  and  $\sigma_{RL_V}$ ; requires  $v = v_{last}$ ,  $|TS_{local} - TS_V| \leq$ 
    $\Delta_{fresh}$ ,  $ScopeOK = 1$ ,  $CertID_j \notin RevokedVerifierCerts_v$ ,  $RootID_k \notin$ 
    $RevokedRoots_v$ , and  $KeyID_i \notin RevokedUAVKeys_v$ 
7: if any check fails then
8:   abort silently; release no Merkle proof, root record, or authentication
   response
9: else
10:  if Profile P then
11:     $D_i$  scans  $ActiveState_k$  in constant time for  $PID_{k,j} = AuthRef$ ,
    fixing  $j_{cur} = j$ 
12:  else
13:    Verify  $AuthRef = RIDID_i^{app}$  against the active jurisdiction-
    approved transmitter identity
14:    Set  $j_{cur}$  to the active interval index and  $PID_{k,j} \leftarrow$ 
     $ActiveState_k[j_{cur}].PID$ 
15:  end if
16:  if the profile-specific reference is invalid, the request is in
     $ReplayCache$ , or no Profile P entry matches then
17:    abort silently
18:  else
19:    Insert  $(CertID_j, N_V, ReqID, Expiry)$  into  $ReplayCache$ ; fail closed
    if insertion cannot be guaranteed
20:     $D_i$  computes  $(ct, ss) \leftarrow ML-KEM.Encaps(ek_{V_j})$ 
21:    Both parties derive  $K_{sess} \leftarrow KDF(ss \parallel ReqID \parallel$ 
     $H_{256}(ReqAuthWire) \parallel "RID-Auth")$ 
22:     $V_j$  recovers  $ss \leftarrow ML-KEM.Decaps(dk_{V_j}, ct)$  and derives the same
     $K_{sess}$ 
23:  end if
24: end if

```

2) PUF-Rooted Post-Quantum Authentication

The UAV proves that the interval pseudonym selected by the profile-specific request belongs to a TA-authorized Merkle root and that it holds the PUF-derived signing identity bound to that root. The Merkle path establishes membership. The ML-DSA signature binds $AuthRef$, the interval pseudonym, and that membership claim to the fresh verifier request and ML-KEM session.

The signature covers the profile-specific observed reference, current interval pseudonym and index, authorized root, root nonce, flight context, verifier nonce, UAV timestamp, ML-KEM ciphertext, and signed request hash. A response from one request cannot be transferred to another because the verifier nonce, KEM ciphertext, timestamp, and request hash change. If PUF reconstruction or key confirmation fails, the UAV returns no alternate proof. If reconstruction fails, the UAV releases neither an interval secret nor a fallback credential.

I. ROOT EXPIRATION AND REVOCATION

Every authorization record carries its own expiration terms:

Algorithm 6 PUF-Rooted Post-Quantum Authentication

```

1:  $D_i$  retrieves  $\pi_{k,j} \leftarrow \text{ActiveState}_k$ 
2:  $R' \leftarrow \text{MajorityPUF}(\text{ChallengeSchedule}(CSeed, q = 1023), 9)$ 
3:  $K_{PUF} \leftarrow FE.Rec(R', HD)$ ; abort silently if reconstruction fails
4:  $S_1 \leftarrow \text{SHAKE256}_{256}(\text{"PUF-SIGN"} \parallel K_{PUF} \parallel \text{ctx}_i)$ 
5:  $(PK_i, SK_i) \leftarrow ML\text{-DSA.KeyGen}_{\text{internal}}(S_1)$ 
6: if  $H_{256}(PK_i) \neq KC_i$  then
7:   Reacquire the complete response vector and repeat reconstruction and
   key regeneration once
8:   if  $H_{256}(PK_i) \neq KC_i$  after the retry then
9:     Erase reconstructed material and abort silently
10:  end if
11: end if
12:  $TS_D \leftarrow \text{CurrentTime}()$ 
13:  $T_H \leftarrow H_{384}(\text{"RID-UAV-Auth-v1"}, \text{Enc}(\text{AuthRef}, \text{PID}_{k,j_{cur}}, j_{cur}, MR_i^{(k)}, \text{RootNonce}_k, \text{FlightCtx}_k, N_k, TS_D, ct, H_{256}(\text{ReqAuthWire})))$ 
14:  $\sigma_D \leftarrow \text{Sign}_{SK_i}(T_H)$ 
15:  $\text{PayloadWire} \leftarrow \text{Enc}(\text{AuthRef}, \text{PID}_{k,j_{cur}}, j_{cur}, \text{RootNonce}_k, \text{FlightCtx}_k, \pi_{k,j_{cur}}, MR_i^{(k)}, \text{AuthRec}_i^{(k)}, TS_D, \sigma_D)$ 
16:  $D_i$  computes  $N_A$  and  $AD_A$  from Eqs. (11)–(12), then sends canonical
    $\text{RespOuter}$  containing  $ct, N_A, \text{AEAD.Enc}_{K_{sess}}(N_A, \text{PayloadWire}, AD_A)$ 
17:  $D_i$  erases  $R', K_{PUF}, S_1, SK_i$ , and temporary signing material
18:  $V_j$  verifies  $\text{RespOuter}$ , decapsulates  $ct$ , derives  $K_{sess}$ , verifies the AEAD
   tag before decoding  $\text{PayloadWire}$ , verifies  $\text{AuthRec}_i^{(k)}$  under  $PK_{TA}$ , and
   checks the record against  $\text{AuthDB}_V$  and  $RL_v$ 
19:  $V_j$  computes  $L_{k,j_{cur}} \leftarrow H_{384}(0x00, \text{Enc}(\text{PID}_{k,j_{cur}}, j_{cur}, \text{RootNonce}_k, \text{FlightCtx}_k))$ 
20:  $V_j$  checks  $\text{VerifyMerkle}(L_{k,j_{cur}}, \pi_{k,j_{cur}}, MR_i^{(k)}) = 1$ 
21:  $V_j$  recomputes  $T_H$  and accepts iff  $\text{Verify}_{PK_i}(T_H, \sigma_D) = 1$ 

```

$$\text{Validity}_k = \{\text{StartTime}_k, \text{EndTime}_k, \text{MaxIntervals}_k, \text{Region}_k\}. \quad (25)$$

A root passes only if the current time, interval index, verifier scope, and operational region fall inside these bounds. Once a root expires or exhausts its intervals, the UAV selects another authorized record or renews it through Algorithm 2.

The TA distributes a versioned revocation list

$$RL_v = \{v, \text{ThisUpdate}_v, \text{NextUpdate}_v, \text{RevokedRoots}_v, \text{RevokedVerifierCerts}_v, \text{RevokedUAVKeys}_v\}, \quad (26)$$

with the signature

$$\sigma_{RL_v} = \text{Sign}_{SK_{TA}}(H_{256}(\text{Enc}(RL_v))). \quad (27)$$

Each UAV and verifier retains the highest valid list version it has accepted. An authentication exchange is permitted only when the list signature verifies, $v \geq v_{last}$, the current time lies in $[\text{ThisUpdate}_v, \text{NextUpdate}_v]$, the active RootID_k is absent from RevokedRoots_v , CertID_j is absent from $\text{RevokedVerifierCerts}_v$, and KeyID_i is absent from RevokedUAVKeys_v . The accepted value of v becomes the new v_{last} . A device that has no valid list or whose list has passed NextUpdate_v fails closed and does not authenticate.

Revocation-list synchronization is periodic and remains outside the Remote ID broadcast path. Each UAV and verifier validates and caches the TA-signed list independently. The verifier includes only the list version v in its signed authentication request. The UAV accepts the request only when v matches its current valid cache and the verifier certificate, active root, and UAV signing-key identifier are not revoked. The verifier applies the same cached list to the disclosed UAV root before acceptance. This mechanism preserves offline operation

between synchronization intervals without transmitting the revocation list during authentication.

IV. SECURITY ANALYSIS

The analysis separates properties provided by the cryptographic protocol from properties that depend on the physical implementation. The claimed guarantees cover Profile P pseudonymous broadcast, authorized verification in both profiles, lawful traceability, spoofing and replay resistance, verifier-database disclosure, passive nonvolatile-memory disclosure, revocation, and post-quantum authentication. Active control of an intact captured UAV is treated as a successful compromise rather than as an attack prevented by the protocol.

A. THREAT MODEL

The adversary controls the broadcast and unicast networks. It may record Remote ID messages, replay or inject broadcasts, initiate unauthorized sessions, eavesdrop on protected exchanges, modify or drop packets, and attempt man-in-the-middle attacks.

A verifier authorization database may be disclosed. The resulting information includes scoped public keys, TA-signed Merkle-root records, root nonces, contexts, validity intervals, and revocation metadata. The database contains no PUF-derived master secret, signing seed, interval secret, private signing key, or legal operator identity.

Passive nonvolatile-memory disclosure of a powered-off UAV is also included. The adversary obtains $CSeed$, HD , public parameters, root nonces, flight contexts, authorization records, the last consumed interval, and the cached revocation-list version. The claim in this case is limited to the absence of persistent private or PUF-derived secrets from that storage image.

Active control of an intact captured UAV is not resisted by the present construction. An adversary that can boot modified firmware and invoke the PUF acquisition and fuzzy-extractor procedures can reconstruct K_{PUF} , derive S_1 and S_2 , regenerate SK_i , and compute valid Merkle material. Such a device can authenticate as the enrolled UAV until KeyID_i or all active roots are revoked. Secure boot, locked debug interfaces, tamper response, and a hardware-isolated PUF service could narrow this exposure, but no such control is assumed here.

Enrollment and renewal use an authenticated and confidential out-of-band channel. Their channel security is an explicit trust assumption. Volatile-memory extraction during authentication, compromise of the TA signing key or legal-registration database, invasive hardware attacks, and implementation-level side channels are outside the claimed protocol guarantees.

A quantum-capable adversary is considered for public-key cryptography. ML-KEM provides session establishment and ML-DSA provides digital signatures. Hash and KDF output lengths account for quantum search speedups.

B. SECURITY ASSUMPTIONS

The analysis relies on the following assumptions.

TABLE 7. Adversary capabilities and corresponding claim boundaries.

Capability	Exposed information or action	Protocol claim
Broadcast and unicast network control	Pseudonyms, telemetry, replay, injection, modification, and packet loss	Authentication, integrity, freshness, and session secrecy are analyzed under standard cryptographic assumptions.
Verifier-database disclosure	Scoped public authorization material and current revocation metadata	No future pseudonym seed, UAV private key, PUF-derived secret, or operator identity is disclosed.
Passive UAV storage disclosure	Public challenge schedule, helper data, root metadata, authorization records, and persistent counters	No persistent private signing key or extracted PUF secret is present in the storage image.
Active control of an intact UAV	Execution of the PUF and extractor through modified firmware	The attacker can regenerate the UAV credentials; containment depends on UAV-key, root, and verifier-certificate revocation.
Excluded implementation attacks	Volatile-memory extraction, invasive probing, side channels, and TA compromise	No protocol-level resistance is claimed.

- **A1: Hash and KDF security.** H_{384} is collision and second-preimage resistant for Merkle and transcript binding; H_{256} is preimage resistant for pseudonym generation and behaves as a secure checksum or public-key confirmation hash in its stated uses. The KDF behaves as a pseudorandom function under domain-separated, canonically encoded inputs.
- **A2: Deployment RO-PUF entropy and device specificity.** A physical deployment must establish that the majority-voted response vector has the required conditional min-entropy, remains reconstructable over the deployment environment, and is unpredictable without access to the enrolled device. The HSpice study in Section V-E supplies model-specific co-simulation evidence only; it does not establish the physical-device part of this assumption.
- **A3: Fuzzy-extractor correctness.** For response vectors within the correction radius of BCH(1023, 943, 17), $FE.Rec$ reconstructs the same K_{PUF} produced at enrollment. Public helper data are included in the entropy budget, and decoding failure causes an abort.
- **A4: Passive storage-disclosure boundary.** The adversary may read nonvolatile memory but does not execute the PUF reconstruction procedure as part of the passive-disclosure claim. Active control of an intact device violates this assumption and permits credential regeneration.
- **A5: Merkle-tree integrity.** A valid authentication path for a leaf outside the authorized tree cannot be produced without a collision or second preimage in the underlying hash construction.
- **A6: Post-quantum primitive security.** ML-KEM is IND-CCA secure and ML-DSA is existentially unforgeable under chosen-message attack for the selected parameter sets.
- **A7: AEAD security.** The AEAD construction provides confidentiality and integrity under a fresh session key.
- **A8: Temporary-state erasure.** PUF-derived secrets, regenerated private keys, root seeds, and active authentication material are erased after their intended use.

cation material are erased after their intended use.

- **A9: Enrollment-channel security.** Enrollment and renewal are performed through an authenticated and confidential out-of-band channel. Compromise of that channel or of the TA is outside the authentication proof.
- **A10: Revocation freshness.** A UAV or verifier authenticates only with a valid TA-signed revocation list whose version is no lower than the highest version retained locally and whose $NextUpdate_v$ has not passed.
- **A11: Crash-consistent interval reservation.** $ReserveNext$ returns success only after the new journal generation is durable and recoverable. The UAV transmits no packet containing a newly selected pseudonym before this commit completes. A failed or indeterminate commit suppresses transmission.
- **A12: Canonical parsing and state freshness.** All cryptographic inputs use $Enc(\cdot)$; noncanonical messages are rejected. Accepted requests are inserted into a fail-closed replay cache, the local monotonic clock cannot roll back without detection, and scope authorization is evaluated by Eq. (15).

C. SECURITY GOALS

The protocol is designed to provide the following properties within the threat model above.

- **Identifier-level broadcast pseudonymity in Profile P:** a passive observer without S_2 or $Seed_k$ cannot derive a long-term identity or cryptographic linkage from the rotating identifiers alone. Profile S does not claim this property because it retains the jurisdiction-approved identifier.
- **Conditional traceability:** the TA can associate a verified Merkle root with the enrolled UAV and legal identity under authorized conditions.
- **Verifier-restricted proof disclosure:** Merkle paths, root records, signatures, and authentication payloads are released only after verifier authentication and session-key establishment.
- **UAV impersonation resistance:** an adversary without

the regenerated ML-DSA signing key cannot produce a valid transcript-bound proof for an uncompromised UAV.

- **Replay protection:** previously recorded authentication messages cannot be reused as fresh evidence.
- **Verifier-database disclosure resistance:** disclosure of $AuthDB_{V_j}$ does not reveal future pseudonym seeds, PUF-derived secrets, private signing keys, or operator identity.
- **Passive nonvolatile-memory disclosure resistance:** a storage image of a powered-off UAV contains no persistent private signing key or extracted PUF secret.
- **Revocation enforcement:** expired, revoked, or stale authorization state is rejected when the fail-closed revocation-list conditions of Section III-I hold.
- **Crash-recovery pseudonym non-reuse:** under Assumption A11, a pseudonym reserved for one rotation interval is never assigned to a later interval after restart. Crashes may consume unused indices, so gap-free interval use is not claimed.
- **Post-quantum authentication:** ML-KEM-768 and ML-DSA-65 provide category-3 key-establishment and signature security, respectively; no uniform category-3 claim is made for all protocol properties.

D. PROPERTY-DEPENDENT SECURITY STRENGTH

The NIST categories assigned to ML-KEM-768 and ML-DSA-65 apply to those primitives and do not automatically characterize the complete protocol. The construction therefore states separate bounds for separate properties. Session establishment and signature forgery rely on the category-3 ML-KEM-768 and ML-DSA-65 parameter sets. Merkle-root and transcript binding use 384-bit TupleHash outputs, giving a 192-bit classical generic collision bound. The 128-bit PID is a compact public handle: it provides a 2^{128} preimage space but only a 64-bit birthday scale for arbitrary collisions, which is why duplicate identifiers are detected and rejected during root construction. The fuzzy extractor uses a target statistical distance $\epsilon_{FE} = 2^{-64}$ and is not described as category 3. The present entropy estimate is derived from the modeled HSpice response matrix; the same bound must be established independently for any fabricated deployment PUF. Consequently, the protocol has no single uniform security-category label; each security claim is bounded by the primitive, parameter, and evidence source relevant to that claim.

E. SECURITY OF MERKLE-ROOT AUTHORIZATION

During enrollment or renewal, UAV D_i derives a PUF-rooted seed for root k :

$$Seed_k = KDF(S_2 \parallel RootNonce_k \parallel k \parallel \text{"MerkleRoot"}). \quad (28)$$

For each interval j , the UAV derives:

$$X_{k,j} = KDF(Seed_k \parallel j \parallel \text{"IntervalSecret"}), \quad (29)$$

$$PID_{k,j} = Trunc_{128}(H_{256}(\text{"RID"} \parallel X_{k,j} \parallel j \parallel RootNonce_k \parallel FlightCtx_k)), \quad (30)$$

and:

$$L_{k,j} = H_{384}(0x00, Enc(PID_{k,j}, j, RootNonce_k, FlightCtx_k)). \quad (31)$$

The TA signs the root authorization record:

$$AuthRec_i^{(k)} = \text{Sign}_{SK_{TA}}(PK_i \parallel MR_i^{(k)} \parallel RootNonce_k \parallel FlightCtx_k \parallel Validity_k \parallel Scope_k). \quad (32)$$

This signature binds the Merkle root to the UAV public key, nonce, authorization context, validity period, and scope. A verifier cannot replace $MR_i^{(k)}$, extend its validity, change its scope, or bind it to another UAV without forging a TA signature. A forged Merkle proof for a pseudonym outside the authorized tree would require a hash collision or a second-preimage attack against the Merkle construction.

The TA may distribute $MR_i^{(k)}$, $RootNonce_k$, $FlightCtx_k$, and $AuthRec_i^{(k)}$ to authorized verifiers. These values do not allow prediction of $PID_{k,j}$ because the derivation also depends on S_2 and $Seed_k$, which are never distributed. A verifier can validate a disclosed proof, but it cannot compute the undisclosed broadcast sequence.

F. BROADCAST PSEUDONYMITY AND LINKABILITY

Under Profile P, the complete jurisdiction-required message set includes a Basic ID message whose UAS-ID field is

$$BasicID_j.UASID = SSIType \parallel PID_{k,j} \parallel 0^{24}. \quad (33)$$

The remaining mandatory identity, location, system, operator, status, timing, and jurisdiction-specific fields are transmitted by the standards-conforming RID stack. Under Profile S, $BasicID_j.UASID = RIDID_i^{app}$ and the pseudonym is not part of routine broadcast.

The broadcast does not include:

$$\mathcal{N}_{broadcast} = \{MR_i^{(k)}, \pi_{k,j}, RootNonce_k, S_2, Seed_k, X_{k,j}, SK_i\}. \quad (34)$$

Under the pseudorandomness of the KDF and the preimage resistance of H_{256} , the sequence:

$$PID_{k,1}, PID_{k,2}, \dots, PID_{k,n} \quad (35)$$

contains no efficiently computable identifier relation for a Profile P observer without S_2 or $Seed_k$. The Merkle root and authentication path are absent from the broadcast, so an observer cannot test root membership from Remote ID packets alone. This separation between rotating public identifiers and authorized resolution follows the design principle used by ARID, A2RID, and pqRID [17]–[19].

The claim is limited to identifier-level cryptographic linkage under the stated KDF assumption. It is not a formal observational-equivalence result. Location, velocity, radio fingerprints, and continuous trajectory observations can still correlate successive broadcasts, as documented in the Remote ID privacy literature [1], [2].

G. CONDITIONAL TRACEABILITY

The TA stores the protected legal mapping:

$$MR_i^{(k)} \rightarrow ID_i. \quad (36)$$

When a verifier receives a protected authentication payload containing $PID_{k,j}$ and $\pi_{k,j}$, it computes $L_{k,j}$ and checks:

$$\text{VerifyMerkle}(L_{k,j}, \pi_{k,j}, MR_i^{(k)}) = 1. \quad (37)$$

It also verifies $\text{AuthRec}_i^{(k)}$ using PK_{TA} . A valid proof shows that the disclosed pseudonym belongs to a TA-authorized root. The verifier may learn that the event is associated with a valid enrolled UAV inside its authorization scope. The operator identity remains hidden unless the TA performs lawful traceability using the legal registration database.

This separation is central to Profile P: public receivers see pseudonyms, authorized verifiers validate roots, and the TA retains the legal identity mapping. In Profile S, public receivers instead see the approved Remote ID identity; the protected exchange supplies supplementary authentication without a broadcast-privacy claim.

H. VERIFIER AUTHENTICATION AND SESSION-KEY SECURITY

The UAV releases a Merkle path, root record, or signature only after authenticating the verifier. The request is

$$\text{ReqAuthWire} = \text{Enc}(\text{Cert}_V, ek_V, \text{AuthRef}, N_V, TS_V, \text{RequestedScope}, v, \text{RIDSetDigest})^{\sigma_D} = \text{Sign}_{SK_i}(T_H). \quad (38)$$

The verifier signs the complete request:

$$\sigma_V = \text{Sign}_{SK_V}(H_{256}(\text{ReqAuthWire})). \quad (39)$$

The UAV first enforces canonical decoding. It then verifies the TA-issued certificate, verifier signature, nonce-cache freshness, rollback-protected clock window, set-intersection scope predicate, cached revocation-list signature, and exact requested list version. It rejects the request when the verifier-certificate identifier, active-root identifier, or UAV signing-key identifier appears in the cached list. Under Profile P it resolves the raw broadcast pseudonym through the constant-time scan in Algorithm 5; under Profile S it verifies the approved identifier against the active transmitter identity. Requests that fail any check produce no response.

After verifier acceptance, the UAV computes

$$(ct, ss) \leftarrow \text{ML-KEM}.\text{Encaps}(ek_V), \quad (40)$$

and both parties derive

$$K_{\text{sess}} = \text{KDF}(ss \parallel \text{ReqID} \parallel H_{256}(\text{ReqAuthWire}) \parallel \text{"RID-Auth"}). \quad (41)$$

A man-in-the-middle cannot alter ek_V , AuthRef , RequestedScope , v , or N_V without invalidating σ_V . A stale or repeated request is rejected by the rollback-protected clock window and the $(\text{CertID}_j, N_V, \text{ReqID})$ replay cache. Under the IND-CCA security of ML-KEM and the pseudorandomness of the KDF,

the session key is hidden from an eavesdropper. For an adversary $\mathcal{A}$ attempting to distinguish K_{sess} from random,

$$\text{Adv}_{\mathcal{A}}^{\text{sess}} \leq \text{Adv}_{\mathcal{A}}^{\text{ML-KEM}} + \text{Adv}_{\mathcal{A}}^{\text{KDF}} + \text{Adv}_{\mathcal{A}}^{\text{fresh}}, \quad (42)$$

where $\text{Adv}_{\mathcal{A}}^{\text{fresh}}$ represents nonce or timestamp failure.

I. PUF-ROOTED POST-QUANTUM AUTHENTICATION

The on-demand proof combines Merkle membership with a PUF-regenerated ML-DSA signature. The UAV reconstructs

$$C = \text{ChallengeSchedule}(C\text{Seed}, q = 1023), \quad (43)$$

$$R' = \text{MajorityPUF}(C, 9), \quad (44)$$

$$K_{\text{PUF}} = \text{FE}.\text{Rec}(R', HD), \quad (45)$$

$$S_1 = \text{SHAKE256}_{256}(\text{"PUF-SIGN"} \parallel K_{\text{PUF}} \parallel \text{ctx}_i), \quad (46)$$

$$\parallel \text{ctx}_i), \quad (47)$$

$$(PK_i, SK_i) \leftarrow \text{ML-DSA}.\text{KeyGen}_{\text{internal}}(S_1). \quad (48)$$

The key-confirmation check $H_{256}(PK_i) = KC_i$ prevents the UAV from signing with a key produced by failed reconstruction. Failure after one complete reacquisition terminates the exchange without releasing an alternate credential.

The transcript hash is

$$T_H = H_{384}(\text{"RID-UAV-Auth-v1"}, \text{Enc}(\text{AuthRef}, PID_{k,j_{\text{cur}}}, j_{\text{cur}}, MR_i^{(k)}), \text{FlightCtx}_k, N_V, TS_D, ct, H_{256}(\text{ReqAuthWire})). \quad (49)$$

The UAV signs

$$\sigma_D = \text{Sign}_{SK_i}(T_H). \quad (50)$$

The verifier accepts only if the TA authorization record is valid, the disclosed leaf verifies against $MR_i^{(k)}$, and

$$\text{Verify}_{PK_i}(T_H, \sigma_D) = 1. \quad (51)$$

The fresh request, nonce, timestamp, KEM ciphertext, root, and flight context prevent replay or transfer across sessions and roots. No $X_{k,j}$ value, root seed, extractor output, or private key is disclosed.

Assuming ML-DSA unforgeability, secrecy of the PUF-derived signing seed, and secure hashing and key derivation,

$$\text{Adv}_{\mathcal{A}}^{\text{auth}} \leq \text{Adv}_{\mathcal{A}}^{\text{ML-DSA}} + \text{Adv}_{\mathcal{A}}^{\text{PUF}} + \text{Adv}_{\mathcal{A}}^{\text{KDF}} + \text{Adv}_{\mathcal{A}}^{\text{Hash}}. \quad (52)$$

Under the stated assumptions, this advantage is negligible.

J. SYMBOLIC MODEL AND CLAIM BOUNDARY

The supplementary Tamarin theory specifies the Profile P on-demand authentication exchange under the Dolev–Yao network model and the standard symbolic abstractions for public-key encryption, signatures, symmetric encryption, and hashing [40]. ML-KEM encapsulation is represented by encryption of a fresh shared secret to the verifier's ephemeral public key, followed by decapsulation with the matching private key. ML-DSA is represented by an unforgeable signature constructor.

Enrollment is represented as an authenticated and confidential out-of-band operation. The model contains two independently authorized Merkle roots, four leaves per root, ordered

two-level authentication paths, and explicit leaf indices. This structure permits wrong-leaf, wrong-index, sibling-order, and cross-root substitutions to be distinguished without expanding the model to the deployment value of n .

The current revocation-list version is included in every authentication request, while the signed list itself is maintained as local state. Verifier acceptance requires the current list version, a valid TA signature, and non-revoked root and verifier states. The list is represented by a monotonically changing epoch rather than by real-valued time. Absence of a current valid epoch prevents the acceptance rule from firing, which captures fail-closed operation.

Separate compromise rules disclose the UAV's nonvolatile storage image and the verifier's authorization database. These rules reveal challenge-schedule data, helper data, root meta-data, authorization records, public keys, and revocation state, but not the reconstructed PUF master secret or the regenerated signing key. The positive authentication lemmas are conditional on the absence of active captured-device control. A distinct capture rule releases the reconstructed PUF secret to represent modified firmware invoking the PUF and extractor; no resistance claim is made after that event.

The theory contains trace lemmas for verifier-reaching executability, ML-KEM session-key secrecy, injective agreement on the complete transcript, Merkle-root and leaf-index binding, revocation-state agreement, and preservation of authentication after passive storage or verifier-database disclosure. These trace properties do not establish RO-PUF entropy, fuzzy-extractor reliability, silicon tamper resistance, Profile P broadcast unlinkability, or the Profile S RID-message digest binding. Pseudonym unlinkability is supported by the analytical argument in Section IV-F and the cited Remote ID literature; it is not claimed as a Tamarin-verified property.

K. SECURITY EFFECTS OF REPRESENTATIVE ATTACKS

Table 9 distinguishes attacks prevented by the protocol from attacks that define its deployment boundary.

L. LIMITATIONS OF THE SECURITY CLAIMS

Identifier-level pseudonymity does not hide physical movement. A receiver that continuously observes position, altitude, velocity, timing, or radio characteristics may correlate successive broadcasts. No formal equivalence proof is claimed for unlinkability.

The byte-level protocol assumes strict canonical decoding, fail-closed replay-cache insertion, and rollback-protected time. Implementations that accept alternative encodings, evict unexpired replay entries, reuse an AEAD nonce under one session key, or compare scope strings without semantic set validation fall outside the security claims. Revocation of $KeyID_i$ disables the PUF-regenerated signing identity across all roots; root-only revocation is insufficient after confirmed active-device capture.

The UAV authentication result consists of a transcript-bound ML-DSA signature and a Merkle membership proof. No interval secret is disclosed. Evidentiary use still depends

on certificate status, revocation-list freshness, root validity, timestamp quality, verifier logging, and preservation of the authenticated transcript.

The standards mapping does not establish regulatory certification. Profile P requires an accepted Session-ID/SSI subtype and applicable means of compliance; Profile S preserves the approved identifier but does not provide pseudonymous public broadcast. Neither profile implements RFC 9575 DRIP Broadcast Authentication, and the host-side prototype does not establish BLE/Wi-Fi frame conformance or interoperability with an independent accepted receiver.

Active control of an intact captured UAV defeats the PUF-derived identity because the attacker can invoke the same PUF reconstruction procedure as legitimate firmware. The protocol protects against passive storage disclosure, not against a captured device that remains operational under adversarial software. Secure boot, debug locking, hardware isolation, and tamper response are deployment measures rather than properties of the protocol analyzed here.

The TA remains a trust anchor. Compromise of SK_{TA} or the legal mapping database affects root authorization and traceability. Key rotation, threshold authorization, access control, audit logging, and separation between legal and operational databases are required in deployment.

PUF security depends on response entropy, environmental reliability, aging behavior, helper-data leakage, response correlation, and modeling resistance. Circuit simulation does not establish invasive-hardware resistance or fabricated-silicon behavior.

Revocation is bounded by the validity period of the latest signed list. A UAV or verifier fails closed after $NextUpdate_v$, so loss of synchronization reduces availability rather than silently accepting stale authorization.

Crash-recovery pseudonym non-reuse depends on the durable semantics of *ReserveNext*. The evaluated implementation uses a preallocated two-slot journal on ext4 and completes a checked `fdatasync()` plus read-back validation before transmission. This ordering enforces the software reservation boundary, but the empirical result remains specific to the tested microSD card, controller, kernel, filesystem, and mount configuration. The protocol permits skipped indices after a crash and does not claim exactly-once delivery or gap-free use. Storage hardware that acknowledges a flush before the data reaches nonvolatile media would require a monotonic counter, transactional nonvolatile memory, or another independently validated persistence mechanism.

M. OPTIONAL DEPLOYMENT HARDENING: DISTRIBUTING TRUST ACROSS THE TRUSTED AUTHORITY

Threshold authorization can reduce the effect of a compromised TA signing key. It is an optional deployment hardening measure rather than a requirement of the base protocol.

Every root authorization in this protocol reduces, in the end, to one signature under SK_{TA} . That is a single point of failure: a stolen or misused TA key can mint a valid $AuthRec_i^{(k)}$ for an unauthorized device, and nothing downstream of Eq. (22)

TABLE 8. Scope of the supplementary symbolic model.

Model element	Representation and claim boundary
Enrollment	Authenticated and confidential out-of-band delivery; channel compromise is not proved.
Merkle authorization	Two roots, four leaves per root, explicit indices, sibling order, and two-level paths.
Revocation	TA-signed monotonic epochs with fail-closed acceptance and explicit revoked-root and revoked-verifier states.
Compromise rules	Passive UAV storage disclosure and verifier-database disclosure are represented explicitly.
Active captured device	PUF reconstruction capability is released; positive authentication claims are conditioned on the absence of this event.
Trace properties	Verifier commit reachability, session secrecy, injective agreement, Merkle binding, and current-revocation-state agreement.
Not formally claimed	PUF entropy, extractor correctness, concrete storage crash consistency, invasive-hardware resistance, side-channel resistance, Profile P pseudonym unlinkability, and Profile S RID-message digest binding.

TABLE 9. Security effects of representative attacks.

Attack	Security effect and control
Broadcast tracking	Rotating identifiers contain no public root or path relation. Identifier-level cryptographic linkage requires S_2 or $Seed_k$. Continuous trajectory and radio correlation remain possible.
Copied Remote ID broadcast	A copied $PID_{k,j}$ can be rebroadcast, but the sender cannot answer a fresh authorized challenge without the matching PUF-regenerated ML-DSA key and Merkle path.
Authentication replay	N_V , $ReqID$, TS_V , TS_D , ct , N_A , $ReplayCache$, and $H_{256}(ReqAuthWire)$ bind the response to one fresh session.
Crash or power loss during pseudonym rotation	The next index is committed before first use. Recovery does not reuse the reserved pseudonym; a crash between commit and transmission may skip one index.
Man-in-the-middle modification	The verifier signs the complete request, ML-KEM establishes the session secret, and AEAD protects the returned root record, path, and signature.
Verifier-database disclosure	The database contains only scoped public authorization material. It permits recognition of roots already disclosed to that verifier but does not reveal future pseudonyms, PUF-derived secrets, private keys, or operator identity.
Passive UAV storage disclosure	The storage image contains public reconstruction and authorization data but no K_{PUF} , S_1 , S_2 , $Seed_k$, $X_{k,j}$, or SK_i .
Active captured-device control	Modified firmware can invoke the PUF and extractor, regenerate the signing identity, and authenticate as the UAV. The protocol does not prevent this attack; containment requires prompt revocation of the affected roots.
Merkle substitution	The TA signature binds PK_i , $MR_i^{(k)}$, context, validity, and scope. The verifier recomputes the ordered path and rejects wrong-leaf, wrong-index, sibling-order, and cross-root substitutions.
Stale or revoked authorization	Authentication fails when RL_v is unsigned, older than the locally retained version, expired, or lists the root or verifier certificate as revoked.
Quantum-capable cryptanalysis	Session establishment and signatures use ML-KEM and ML-DSA rather than RSA or elliptic-curve primitives.
Denial of service	Jamming, packet dropping, and request flooding remain possible. Certificate, signature, scope, freshness, and revocation checks precede PUF acquisition and signing, but rate limiting and radio-layer mitigation remain implementation controls.

would catch it. Removing the TA from the real-time path, which Phase 3 already does, only addresses availability. It does nothing for key compromise, and the base protocol as described in Section III does not either.

The fix does not need new cryptographic assumptions. Replace the single TA signer with n independent shareholders, each running ordinary $ML\text{-}DSA.KeyGen()$,

$$(PK_{TA,\ell}, SK_{TA,\ell}) \leftarrow ML\text{-}DSA.KeyGen(), \quad \ell = 1, \dots, n. \quad (53)$$

and publish a quorum policy $QuorumPolicy =$

$\{t, n, \{PK_{TA,1}, \dots, PK_{TA,n}\}\}$ alongside $Params$ at system initialization. A root authorization record is no longer a single signature but a set of t signatures over the same payload, collected from any t of the n shareholders,

$$AuthRec_i^{(k)} = \{\sigma_{TA,\ell}\}_{\ell \in S}, \quad |S| = t, \quad (54)$$

where each $\sigma_{TA,\ell}$ signs the identical payload of Eq. (22) under $SK_{TA,\ell}$. A verifier or UAV accepts the record only if at least t of the n signatures check out under distinct shareholder keys. No fewer than t compromised or colluding shareholders

can produce a valid record on their own, which is exactly the guarantee a single-key TA cannot offer.

This costs bandwidth, not new machinery: $\text{AuthRec}_i^{(k)}$ grows from one ML-DSA-65 signature to t of them, roughly $t \times 3309$ bytes against the parameter choice fixed in Phase 1. That growth lands entirely in the TA-to-verifier authorization database and the UAV's own nonvolatile storage, both off the broadcast path Phase 3 keeps compact, so it does not touch the constraint the rest of the protocol is built around. Deployments that do not need this hardening can keep the single-signer TA of the base protocol unchanged; the quorum construction is an additive option, not a prerequisite for the rest of the security analysis in this section.

A cryptographically tighter alternative exists, and is worth naming rather than glossing over. Recent constructions produce a genuine threshold ML-DSA signature, one object indistinguishable from a single signer's output, without ever reconstructing SK_{TA} : del Pino and Niot give a threshold scheme for $t \leq 8$ parties with signature size close to a single Dilithium signature [41], and Dufka *et al.* give a universally composable two-party distributed ML-DSA signing protocol [42]. Both remove the $O(t)$ growth in Eq. (54) at the cost of a multi-round signing protocol among shareholders, since Dilithium's rejection-sampling step is harder to run under multiparty computation than the analogous step in Schnorr-style threshold signatures. We treat the quorum construction above as the deployable answer today and this line of work as the natural upgrade once threshold ML-DSA implementations mature past small party counts.

V. PERFORMANCE EVALUATION

This section evaluates the computational, communication, and storage cost of the proposed protocol. The analysis follows the four protocol phases: system initialization, enrollment/renewal, Remote ID broadcast, and on-demand in-flight authentication. Expensive cryptographic material is kept outside routine Remote ID broadcast. Merkle-root authorization, verifier certification, and root distribution are handled before operation. Under Profile P, the broadcast adds a 16-byte pseudonym inside the standards-mapped SSI field; under Profile S, no cryptographic field is added to the jurisdiction-required broadcast. Proof material is transmitted only during authenticated unicast verification.

Table 10 gives the numbers the rest of this section works up to, measured on the Raspberry Pi 5 setup described in Section V-B, before the detailed cost model and per-phase breakdown that follow.

The evaluation separates persistent costs from per-flight and per-request costs. Enrollment and Merkle-root authorization are performed infrequently. Local root activation is performed before flight without contacting the Trusted Authority (TA). Broadcast cost is incurred at every Remote ID interval. On-demand authentication cost is incurred only when an authorized verifier requests proof.

TABLE 10. Headline measured results, ahead of the detailed evaluation below.

Quantity	Result
Merkle-root generation, $n = 1024$	5.4 ms
Merkle-root generation, $n = 16384$	89.2 ms
Pre-flight root activation, $n = 1024$	6.1 ms
Pre-flight root activation, $n = 16384$	89.5 ms
On-demand authentication, $n = 1024$	[measure]
On-demand authentication, $n = 16384$	[measure]
Broadcast crypto payload	16 bytes, constant in n

A. EVALUATION SCOPE

The following components were evaluated:

- enrollment and renewal overhead for generating PUF-derived Merkle roots and obtaining TA-signed authorization records;
- verifier enrollment overhead and authorization database distribution;
- local pre-flight activation cost for reconstructing the active Merkle tree and preparing volatile authentication paths;
- routine Remote ID broadcast overhead;
- PUF-rooted post-quantum authentication overhead, including the $O(n)$ pseudonym lookup;
- UAV, TA, and verifier storage requirements.

Numerical latencies for cryptographic software, Merkle processing, journaling, and fuzzy-extractor execution are obtained from repeated measurements on the Raspberry Pi 5. RO-PUF response generation, process variation, voltage and temperature dependence, and comparator noise are obtained from HSpice simulation. Quantities that combine simulated response generation with Raspberry Pi processing are identified as co-simulation results; no PUF entropy, reliability, or acquisition result is described as a measurement from a fabricated device.

B. IMPLEMENTATION PLATFORM AND MEASUREMENT METHOD

The protocol was evaluated on a Raspberry Pi 5, acting as the UAV, with the Trusted Authority and verifier roles run on a separate machine and connected only through the file exchange described below, not a live network link. Table 11 lists the platform and library versions actually used.

The interval journal is created and preallocated during system initialization, so routine reservations do not alter the file length or directory entry. Each slot stores a magic value, format version, root identifier, generation number, next interval index, and checksum. To reserve interval j , the implementation writes a complete record containing $j + 1$ to the inactive slot with `pwrite()`, calls `fdatasync()`, checks the return status, reads the slot back, and verifies its generation number and checksum. The broadcast routine receives a success result only after these steps complete. No packet containing

TABLE 11. Circuit–software co-simulation and Raspberry Pi measurement setup.

Parameter	Configuration
Embedded platform	Raspberry Pi 5
Processor	Broadcom BCM2712, quad-core Arm Cortex-A76 @ 2.4 GHz
Memory	4 GB LPDDR4X
Nonvolatile medium	SanDisk Extreme 64 GB microSD card, UHS-I, Class 10
Filesystem	ext4 with journaling enabled; mounted with <code>defaults,noatime,data=ordered,barrier=1,errors=remount-ro</code>
Interval journal	One preallocated 8192-byte file containing two alternating 4096-byte generation-numbered slots
Reservation write path	Complete inactive-slot update by <code>pwrite()</code> , followed by checked <code>fdatsync()</code> and read-back validation before first transmission
Recovery rule	Select the checksum-valid slot with the highest generation number; fail closed if neither slot validates
Operating system	Raspberry Pi OS (64-bit), Linux, Python 3.11
Implementation language	Python 3.11, with cryptographic primitives called through native C libraries (see below), not implemented in pure Python
Compiler / library	liboqs 0.16.0 (ML-KEM, ML-DSA), official Ascon reference implementation (<code>ascon-c</code> , NIST SP 800-232, compiled locally via <code>gcc</code>), <code>bchlib</code> 2.1.3 (BCH codec)
PUF evidence source	HSpice-generated response matrices for the modeled 45 nm RO-PUF; no physical RO-PUF is connected to the Raspberry Pi
RO-PUF architecture	128 seven-stage ring oscillators arranged in two banks of 64; one 64:1 multiplexer per bank, dual frequency counters, and one comparator; a 12-bit challenge selects one oscillator from each bank and produces one response bit
Response-vector construction	$q = 1023$ distinct challenges generated from <i>CSeed</i> ; nine simulated comparator readings per challenge followed by majority voting; 1023-bit response vector
Process-variation model	HSpice Monte Carlo analysis, number of independent simulated process instances [simulation result]; threshold-voltage and carrier-mobility distributions, correlation assumptions, netlist, model version, and random seeds reported with the dataset
PVT evaluation range	Simulated temperature -40°C to 100°C and supply voltage 0.9 V to 1.1 V
Fuzzy extractor	Syndrome secure sketch based on binary BCH(1023, 943, 17), $t = 8$, with an 80-bit syndrome; seeded 2-universal Toeplitz extractor with 256-bit output and 1278-bit public extractor seed
Simulation entropy criterion	Model-specific lower confidence bound $H_{\infty}^{LB}(R) \geq 464$ bits before helper-data disclosure; at least 384 bits after the 80-bit syndrome leakage, yielding $\epsilon_{FE} \leq 2^{-64}$ rather than a category-3 or fabricated-device entropy claim
Physical-device evidence	None; fabricated-silicon entropy, physical aging, layout-dependent coupling, device-level environmental reliability, invasive resistance, and physical side channels are outside the evaluated evidence
Hash and KDF	Ascon-Hash256 for PID generation, request hashing, checksums, and KDF calls; TupleHash256 with 384-bit output for Merkle leaves, internal nodes, roots, and transcript hashing; SHAKE256 for the two 256-bit PUF-derived domain secrets
Post-quantum KEM	ML-KEM-768
Post-quantum signature	ML-DSA-65
AEAD	Ascon-AEAD128 (NIST SP 800-232)
Merkle hash length	48 bytes (TupleHash256, 384-bit output)
Pseudonym length	16 bytes (128-bit, Trunc_{128})
Intervals per root	$n \in \{1024, 16384\}$
Authorized roots	$m = 2$ (enrollment/renewal testing)
Raspberry Pi trials per software operation	30

$PID_{k,j}$ is transmitted after a failed or indeterminate persistence operation. This sequence uses the data-durability semantics of `fdatsync()` and avoids per-reservation file replacement or directory updates [43], [44].

C. CIRCUIT–SOFTWARE CO-SIMULATION BOUNDARY AND REPRODUCIBILITY

The evaluation uses two evidence sources that are kept separate. HSpice produces raw comparator outcomes for the modeled RO-PUF, indexed by simulated process instance, challenge, repeated read, supply voltage, temperature, and noise realization. These records are exported without majority voting or entropy preprocessing. The Raspberry Pi imports the raw records, applies the nine-read majority rule, executes BCH enrollment or reconstruction, performs Toeplitz extraction, and runs the Merkle and post-quantum protocol operations. A simulated process instance is therefore an independent draw from the stated transistor-variation model, not a fabricated

chip or FPGA device.

Table 12 states the conclusions supported by each evidence source. Raspberry Pi timings support processor-level implementation costs for the supplied response data. HSpice results support only the behavior predicted by the adopted netlist, transistor model, variation distributions, PVT grid, and noise assumptions. Neither source demonstrates manufacturing variation in physical samples, long-term drift, aging, electromagnetic or power side channels, fault injection, invasive probing, or modeling resistance of a deployed device.

The reproducibility package records the HSpice version, technology-model identifier, complete netlist, transistor dimensions, counter window, comparator model, variation distributions and correlations, PVT points, random seeds, and number of simulated instances. The exported dataset retains one row per comparator read with the fields {instance, challenge, repetition, voltage, temperature, noise seed, counter values, comparator bit}. Analysis scripts reconstruct the majority-

TABLE 12. Evidence sources and claim boundary.

Evidence source	Supported conclusion	Not established
HSpipe Monte Carlo	Model-specific response bias, inter-instance variation, PVT sensitivity, simulated BER, and modeled entropy estimate	Fabricated-device entropy, chip uniqueness, physical aging, layout parasitics, and attack resistance
Raspberry Pi 5	Software execution time, memory use, fuzzy-extractor behavior on supplied vectors, and protocol processing cost	Physical RO-PUF acquisition timing, hardware energy, silicon reliability, and hardware tamper resistance
Combined pipeline	Functional compatibility between the specified PUF interface, fuzzy extractor, and authentication protocol	A fabricated hardware-root implementation

TABLE 13. Crash-recovery validation under forced power interruption.

Metric	Result
Forced interruptions	[measure]
Invalid single-slot records	[measure]
Cases with both slots invalid	[measure]
Previously transmitted pseudonyms reused	[measure]
Unused indices consumed	[measure]
Maximum recovery time	[measure]

voted vectors and report every exclusion, failed simulation, and decoder failure. Raspberry Pi source code, compiler and library versions, command lines, processor settings, and raw timing samples are retained separately from the circuit data.

D. CRASH-RECOVERY VALIDATION

Crash recovery is evaluated by interrupting power at four locations in the reservation–transmission sequence: before the slot write, during the write, after `fdatasync()` returns but before transmission, and immediately after transmission. After each restart, both slots are read independently and the valid record with the highest generation number is selected. The test records the recovered generation, recovered next index, checksum status of each slot, whether a previously transmitted pseudonym is selected again, and whether an unused index is consumed.

The acceptance criterion is zero reuse of a pseudonym that had already been transmitted. A power interruption after a durable reservation but before the first packet may consume the reserved index; such an event is counted as a skipped index rather than a recovery failure. Results must be reported for the stated SanDisk Extreme microSD and ext4 configuration because UHS-I and Class 10 ratings specify transfer performance, not power-loss protection.

For each operation op , the mean latency is:

$$T_{op}^{avg} = \frac{1}{r} \sum_{\ell=1}^r T_{op}^{(\ell)}, \quad (55)$$

where r is the number of repeated trials. The standard deviation, minimum, and maximum values should also be reported.

E. HSPICE RO-PUF CO-SIMULATION AND ENTROPY ANALYSIS

The modeled RO-PUF contains 128 seven-stage ring oscillators arranged in two banks of 64. A 12-bit challenge uses six bits to select one oscillator from each bank. The two simulated counter values are compared once, so each challenge produces one comparator bit. A public schedule generated from *CSeed* contains 1023 distinct challenges. Nine comparator reads are generated for each challenge and reduced by majority vote to produce the 1023-bit vector R . The complete vector, rather than one challenge response, is the input to the fuzzy extractor. BCH(1023, 943, 17) corrects at most eight response errors and exposes an 80-bit syndrome.

Each Monte Carlo draw is treated as one simulated process instance. The HSpice characterization reports the technology-model identifier, oscillator and counter netlists, transistor dimensions, modeled threshold-voltage and mobility distributions, within-instance and between-instance correlation assumptions, voltage and temperature grid, counter window, temporal-noise and comparator-jitter models, random seeds, failed runs, and the number of independent instances. The reported inter-instance Hamming distance, uniformity, bit alias, and BER therefore describe the simulated population only.

The entropy workflow uses the statistical procedures of NIST SP 800-90B as conservative analysis tools rather than as a certification claim [45]. SP 800-90B addresses entropy sources for random-bit generation, whereas this construction uses a reproducible device-specific response. The response matrix is consequently retained with its instance, challenge, repetition, and PVT labels instead of being treated as one unlabeled random stream. Applicable IID tests are reported. When the IID hypothesis is not supported, the non-IID estimators are used, and the minimum applicable estimate is retained. Results are reported separately for each PVT condition, together with a stated one-sided confidence level; the security calculation uses the lowest lower-confidence estimate across conditions. Restart-style analyses use independently initialized Monte Carlo instances and are interpreted as simulator/model diagnostics, not as physical power-cycle evidence.

Uniqueness, uniformity, bit aliasing, and BER are diagnostic metrics and do not by themselves establish extractor entropy. The model-specific security quantity is a lower confidence bound on the min-entropy of the complete response vector after accounting for observed bias and inter-bit dependence. Helper data are treated as public. A conservative bound is

$$H_{\infty}^{LB}(R | HD) \geq H_{\infty}^{LB}(R) - 80, \quad (56)$$

where 80 bits is the BCH syndrome length. The co-simulation instantiation is accepted only when $H_{\infty}^{LB}(R) \geq 464$ bits, leaving at least 384 conditional bits and supporting extraction of a 256-bit value with statistical distance no greater than 2^{-64} under the 2-universal extractor construction [32]. This bound applies only to the adopted simulation distribution. A physical deployment must repeat the entropy, reliability, and helper-data analysis on independently manufactured devices before the same bound can be claimed for deployed hardware.

The co-simulation establishes functional consistency among the modeled RO-PUF interface, fuzzy extractor, and authentication protocol. It does not establish physical unclonability, entropy in fabricated silicon, manufacturing-lot variation, long-term aging, invasive resistance, layout-specific coupling, electromagnetic or power leakage, fault-injection resistance, or silicon-specific side channels. No such property is inferred from the simulated uniqueness, BER, or entropy values.

F. PHASE 1: SYSTEM INITIALIZATION COST

System initialization is executed by the TA. The main operation is TA key generation:

$$(PK_{TA}, SK_{TA}) \leftarrow ML\text{-}DSA.KeyGen(). \quad (57)$$

This cost is not incurred by the UAV during flight. It is performed once when the trust domain is established or when TA keys are rotated. The storage required for this phase is limited to the TA signing key pair and the public protocol parameters, $O_{TA}^{init} = |PK_{TA}| + |SK_{TA}| + |Params|$.

G. PHASE 2: ENROLLMENT AND RENEWAL COST

Phase 2 contains UAV enrollment and Merkle-root authorization, verifier enrollment and authorization database distribution, and Merkle-root renewal.

1) UAV Enrollment and Merkle-Root Authorization

The UAV enrollment cost includes acquisition of one 1023-bit majority-voted response vector, fuzzy-extractor generation, derivation of the signing and Merkle secrets, and construction of m Merkle-root commitments. For one UAV, the enrollment latency is:

$$T_{enroll} = T_{PUFvec} + T_{FE.Gen} + 2T_{SHAKE}^{domain} + T_{ML\text{-}DSA.KeyGen} + mT_{root}. \quad (58)$$

The cost of constructing one Merkle root is:

$$T_{root} = T_{KDF}^{root} + n \left(T_{KDF}^{int} + T_{H_{256}}^{PID} + T_{H_{384}}^{leaf} \right) + (n-1)T_{H_{384}}, \quad (59)$$

where T_{KDF}^{root} is the root-seed derivation time, T_{KDF}^{int} is the interval-secret derivation time, $T_{H_{256}}^{PID}$ is the pseudonym-hash time, $T_{H_{384}}^{leaf}$ is the leaf-hash time, and $(n-1)T_{H_{384}}$ represents 384-bit TupleHash internal-node hashing for a binary tree with n leaves.

Enrollment is offline with respect to flight. The TA signs each root authorization record at a cost of $T_{TA}^{root-auth} = mT_{ML\text{-}DSA.Sign}^{TA}$, and the overall enrollment cost is amortized over the number of authorized roots and the number of intervals per root.

2) Verifier Enrollment and Authorization Database Distribution

Verifier enrollment requires local verifier key generation and TA certificate issuance, a one-time cost of $T_{ver-enroll} = T_{ML\text{-}DSA.KeyGen}^V + T_{ML\text{-}DSA.Sign}^{TA}$.

The TA distributes to verifier V_j only the root records in its authorization scope:

$$AuthDB_{V_j} = \{PK_i, MR_i^{(k)}, RootNonce_k, FlightCtx_k, Validity_k, Scope_k, AuthRec_i^{(k)}\}_{i,k \in Scope_j}. \quad (60)$$

If q_j is the number of UAV-root records in the verifier scope, the verifier database size is:

$$O_{V_j}^{AuthDB} = q_j(|PK_i| + |MR| + |RootNonce| + |FlightCtx| + |Validity| + |Scope| + |AuthRec|). \quad (61)$$

The verifier database contains no $S_1, S_2, Seed_k, X_{k,j}, PID_{k,j}, SK_i$, or operator identity. Its storage cost scales with authorized roots, not with the number of interval pseudonyms under each root.

3) Merkle-Root Renewal

Root renewal uses the same Merkle-root generation procedure as enrollment, but it does not require legal identity re-registration or regeneration of the UAV signing public key unless the UAV registration has changed. The renewal latency for m' new roots is:

$$T_{renew} = T_{PUFvec} + T_{FE.Rec} + T_{SHAKE}^{domain} + m'T_{root} + m'T_{ML\text{-}DSA.Sign}^{TA}. \quad (62)$$

Renewal frequency depends on root validity, mission duration, and pseudonym-rotation policy. A larger m' reduces the frequency of renewal but increases UAV nonvolatile root-record storage and TA authorization database size.

H. PHASE 3: REMOTE ID BROADCAST COST

Phase 3 includes local pre-flight root activation and routine Remote ID broadcasting. The UAV does not contact the TA in this phase.

1) Local Pre-Flight Root Activation

Before flight, the UAV selects one valid root record $AuthRec_i^{(k)}$, reconstructs S_2 , derives $Seed_k$, reconstructs the active Merkle tree, and verifies that the computed root matches the TA-authorized $MR_i^{(k)}$. The activation latency is:

$$T_{act} = T_{PUFvec} + T_{FE.Rec} + T_{SHAKE}^{domain} + T_{KDF}^{root} + n \left(T_{KDF}^{int} + T_{H_{256}}^{PID} + T_{H_{384}}^{leaf} \right) + (n-1)T_{H_{384}} + T_{root-check}. \quad (63)$$

The root check is then a direct comparison, $Root(MT_i^{(k)}) \stackrel{?}{=} MR_i^{(k)}$.

TABLE 14. Modeled RO-PUF response-vector and fuzzy-extractor characterization.

Metric	Co-simulation result	Method / condition
Physical RO-PUF devices	0	Circuit–software co-simulation; no fabricated device
Independent simulated process instances	[simulation result]	Independent Monte Carlo draws from the reported variation model
Response-vector length	1023 bits	1023 distinct 12-bit challenges
Comparator reads per challenge	9	Raw simulated reads retained before majority voting
Inter-instance uniqueness	[simulation result]%	Mean pairwise Hamming distance between simulated response vectors with confidence interval
Uniformity	[simulation result]%	Fraction of ones in each simulated response vector with population distribution
Bit alias	[simulation result]	Per-challenge probability across simulated instances with confidence interval
IID assessment	[simulation result]	Applicable SP 800-90B IID tests; non-IID track used when IID is unsupported
Single-read worst-case simulated BER	[simulation result]%	Full simulated voltage and temperature range relative to enrollment
Post-majority worst-case simulated BER	[simulation result]%	Nine reads per challenge followed by majority voting
BCH decoder-failure probability	[simulation result]	Observed co-simulation failure rate for BCH(1023, 943, 17) across the PVT set
Raw-vector min-entropy lower bound $H_{\infty}^{LB}(R)$	[simulation result] bits	Minimum applicable IID/non-IID estimate with stated one-sided confidence level
Helper-data leakage bound	80 bits	BCH syndrome length
Conditional min-entropy lower bound	[simulation result] bits	Eq. (56); at least 384 bits required for $\epsilon_{FE} \leq 2^{-64}$
Extracted master-secret length	256 bits	Seeded 2-universal Toeplitz extraction

After a successful check, the UAV prepares the active state. The protocol requires availability of $PID_{k,j}$, $L_{k,j}$, and $\pi_{k,j}$ during the flight. The storage format is an implementation choice.

A direct proof-table representation requires:

$$O_{active}^{proof} = n(|PID| + |L| + |\pi|). \quad (64)$$

For a Merkle tree with hash length h and depth:

$$d = \lceil \log_2 n \rceil, \quad (65)$$

the proof size is:

$$|\pi| = dh. \quad (66)$$

Thus:

$$O_{active}^{proof} = n(|PID| + h + dh). \quad (67)$$

A more compact active-tree representation stores the pseudonym array and the Merkle tree nodes:

$$O_{active}^{tree} = n|PID| + (2n - 1)h. \quad (68)$$

This representation avoids storing every full authentication path. During authentication, the UAV retrieves the path from the active tree state by reading the sibling nodes along the leaf-to-root path. It does not rebuild the Merkle tree.

For $|PID| = 16$ bytes and $h = 48$ bytes, Table 15 compares the two volatile storage formats. The pseudonym remains the 16-byte $Trunc_{128}$ output, while every Merkle leaf, internal node, root, and authentication-path element is a 48-byte TupleHash256 output. The values below follow from Eqs. (67) and (68).

The compact tree representation is preferable for memory-constrained UAVs. The proof-table representation has faster proof retrieval but higher volatile memory cost.

TABLE 15. Volatile active-state storage for the selected root.

Active-state format	$n = 1024$	$n = 16384$
Proof-table representation	544 KiB	11.50 MiB
Compact tree representation	112 KiB	1.75 MiB

2) Routine Broadcast Overhead

During interval j , the standards-conforming RID stack broadcasts the complete jurisdiction-required message set $RIDSet_j$. Under Profile P, the Basic ID UAS-ID field is

$$UASID_{k,j}^P = SSIType \parallel PID_{k,j} \parallel 0^{24}, \quad (69)$$

where the cryptographic contribution is $O_{RID}^{crypto} = |PID| = 16$ bytes. The one-octet SSI subtype and three null octets complete the 20-octet UAS-ID field. This mapping fits within the ASTM field capacity but remains conditional on an assigned or accepted SSI subtype and applicable Session-ID rules. Under Profile S, $O_{RID}^{crypto} = 0$ because the approved Remote ID identifier and mandatory messages are transmitted unchanged. In both profiles, telemetry remains in the message types defined by the applicable Remote ID specification rather than being packed into the Basic ID field.

No Merkle root, Merkle proof, root nonce, interval secret, ML-KEM ciphertext, or ML-DSA signature is included in routine broadcast. Before the first use of a new interval pseudonym, however, the UAV performs one crash-consistent journal commit. The protocol-side latency at a pseudonym-rotation boundary is therefore

$$T_{broadcast}^{rotate} = T_{reserve}^{NV} + T_{select}^{PID} + T_{format}^{RID}, \quad (70)$$

where $T_{reserve}^{NV}$ includes the inactive-slot write, persistence barrier or flush, and post-write validation. Ordinary packets repeated within the same rotation interval do not repeat this journal operation.

The Profile P cryptographic payload remains constant in n , while Profile S adds no cryptographic broadcast payload; rotation latency in both profiles depends on the persistence mechanism. Broadcast measurements must include the durable reservation cost; completion of a buffered file-system write alone does not satisfy Assumption A11.

I. PHASE 4: ON-DEMAND IN-FLIGHT AUTHENTICATION COST

Phase 4 authenticates the verifier, performs a constant-time scan of the active pseudonym table, establishes an ML-KEM session, regenerates the UAV signing key, and returns a Merkle proof with an ML-DSA signature.

1) Verifier Authentication and Session Establishment

The verifier request requires

$$T_V^{req} = T_{ML-KEM.KeyGen} + T_{ML-DSA.Sign}^V \quad (71)$$

The UAV-side session cost is

$$\begin{aligned} T_D^{sess} = & T_{CertVerify} + T_{ML-DSA.Verify}^V \\ & + T_{fresh} + T_{scope} + T_{resolve}^{profile} \\ & + T_{ML-KEM.Encaps} + T_{KDF}^{sess} \end{aligned} \quad (72)$$

For Profile P, $T_{resolve}^{profile} = T_{lookup}(n)$ covers the branch-free scan over all n cached pseudonyms. It is independent of the matching position but grows linearly with the root interval count. For Profile S, $T_{resolve}^{profile}$ replaces that scan with verification of $RIDID_i^{app}$ against the active transmitter identity. Authentication latency is therefore reported by profile and, for Profile P, by each evaluated value of n .

The verifier decapsulation and key-derivation cost is

$$T_V^{sess} = T_{ML-KEM.Decaps} + T_{KDF}^{sess} \quad (73)$$

The request size is

$$\begin{aligned} O_{ReqAuthWire} = & |Cert_V| + |ek_V| + |AuthRef| + |N_V| \\ & + |TS_V| + |RequestedScope| + |\sigma_V|. \end{aligned} \quad (74)$$

2) PUF-Rooted Post-Quantum Authentication Cost

The UAV retrieves the current Merkle path from volatile active state; it does not rebuild the tree during authentication. With a proof-table representation, retrieval is approximately constant-time. A compact tree representation reads $d = \log_2 n$ sibling nodes:

$$T_{path}^{tree} = dT_{read} \quad (75)$$

The UAV-side computation is

$$\begin{aligned} T_D^A = & T_D^{sess} + T_{path} + T_{PUFvec} + T_{FE.Rec} \\ & + T_{SHAKE}^{domain} + T_{ML-DSA.KeyGen} \\ & + T_{H384}^{transcript} + T_{ML-DSA.Sign} + T_{AEAD.Enc} \end{aligned} \quad (76)$$

The verifier-side computation is

$$\begin{aligned} T_V^A = & T_V^{sess} + T_{AEAD.Dec} + T_{AuthRecVerify} \\ & + dT_{H384} + T_{H384}^{transcript} + T_{ML-DSA.Verify} \end{aligned} \quad (77)$$

The response size is

$$\begin{aligned} O_A = & |ct| + |PID| + |j| + |RootNonce| + |FlightCtx| \\ & + |\pi| + |MR| + |AuthRec| + |TS_D| + |\sigma_D| \\ & + |Tag_{AEAD}| + O_{hdr} \end{aligned} \quad (78)$$

If the verifier already holds the matching root record, $MR_i^{(k)}$ and $AuthRec_i^{(k)}$ may be replaced by a compact root reference:

$$O_A^{cached} = O_A - |MR| - |AuthRec| + |RootRef|. \quad (79)$$

The end-to-end measurement includes the profile-specific reference-resolution cost, full response-vector acquisition, fuzzy-extractor reconstruction, deterministic ML-DSA key regeneration, signing, proof retrieval, serialization, AEAD processing, and verifier-side checks.

J. COMMUNICATION OVERHEAD SUMMARY

Table 16 summarizes the communication overhead of each operational path.

The broadcast overhead is independent of Merkle depth and post-quantum object size. Authentication overhead grows with $\log_2 n$ through the Merkle path, with n through the branch-free pseudonym lookup, and with the selected ML-KEM and ML-DSA parameter sets.

Component sizes. FIPS 203 and FIPS 204 specify $|PK_i| = |PK_{TA}| = |PK_{V_j}| = 1952$ bytes and $|AuthRec| = |Cert_V| = |\sigma_D| = 3309$ bytes for ML-DSA-65, and $|ek_{V_j}| = 1184$ bytes and $|ct| = 1088$ bytes for ML-KEM-768. The construction uses $h = 48$ bytes for each Merkle leaf, internal node, root, and path element, $|PID| = 16$ bytes, and 170 bytes of helper data. The file-based prototype serializes binary fields as hexadecimal JSON, which is not wire efficient. Raw canonical binary serialization must be used for deployment-oriented message sizes.

K. STORAGE OVERHEAD

1) UAV Nonvolatile Storage

After enrollment or renewal, the UAV stores:

$$\begin{aligned} Store_{D_i}^{NV} = & \{CSeed, HD, ctx_i, Params_i, KC_i, RL_v, \\ & \sigma_{RL_v}, v_{last}, RootNonce_k, FlightCtx_k, \\ & AuthRec_i^{(k)}, k, Validity_k, J_{k,0}, J_{k,1}\}_{k=1}^m \end{aligned} \quad (80)$$

The corresponding nonvolatile storage is:

$$\begin{aligned} O_D^{NV} = & |CSeed| + |HD| + |ctx_i| + |Params_i| + |KC_i| \\ & + |RL_v| + |\sigma_{RL_v}| + |v_{last}| \\ & + m(|RootNonce| + |FlightCtx| + |AuthRec| \\ & + |k| + |Validity| + 2|J|), \end{aligned} \quad (81)$$

where

$$|J| = |g| + |k| + |j_{next}| + |H_{256}|. \quad (82)$$

TABLE 16. Communication overhead by protocol path.

Protocol path	Main transmitted fields	Channel
Routine Remote ID broadcast	$PID_{k,j}$ and telemetry only	Broadcast
Verifier authentication request	$Cert_{V_j}, ek_{V_j}, PID, N_V, TS_V, Scope_j, \sigma_{V_j}$	Unicast
UAV authentication response	ct and an encrypted payload containing $PID_{k,j}, j, RootNonce_k, FlightCtx_k, \pi_{k,j}, MR_i^{(k)}, AuthRec_i^{(k)}, TS_D$, and σ_D	Unicast
Cached UAV authentication response	Same response, with $MR_i^{(k)}$ and $AuthRec_i^{(k)}$ replaced by $RootRef$	Unicast

No persistent $SK_i, S_1, S_2, Seed_k, X_{k,j}$, full leaf table, proof table, or pseudonym-certificate set is stored in nonvolatile memory.

2) UAV Volatile Storage During Flight

During Phase 3, the UAV stores the active state for the selected root. As shown in Table 15, the compact tree representation is substantially smaller than the proof-table representation. The protocol is compatible with either representation. The compact representation is preferred when RAM is limited:

$$O_D^V = O_{active}^{free} = n|PID| + (2n - 1)h. \quad (83)$$

If faster proof retrieval is preferred and memory is available, the proof-table representation may be used:

$$O_D^V = O_{active}^{proof} = n(|PID| + h + dh). \quad (84)$$

3) TA and Verifier Storage

The TA stores the legal mapping from Merkle roots to UAV identity:

$$O_{TA}^{map} = N_{roots}(|MR| + |ID_i| + |Validity| + |Scope| + O_{meta}). \quad (85)$$

where N_{roots} is the total number of authorized root records.

The verifier stores only scoped authorization records:

$$O_V^{AuthDB} = q_i(|PK_i| + |MR| + |RootNonce| + |FlightCtx| + |Validity| + |Scope| + |AuthRec|). \quad (86)$$

Verifier storage grows with the number of authorized root records in scope, not with the number of pseudonyms committed under each root.

L. MEASURED LATENCY RESULTS

Table 17 separates Raspberry Pi software measurements from circuit–software co-simulation quantities for Phases 1–4.

Evaluation caveats. The HSpice row measures simulator time for generating the complete 1023-challenge, nine-read dataset under the stated model and computing environment; it is not a physical RO-PUF acquisition time. The majority-voting and fuzzy-extractor row measures Raspberry Pi processing of the imported raw comparator records. In the analytical cost equations, T_{PUFvec} remains a deployment parameter that must be measured on the selected physical PUF. On-demand-authentication totals reported for the co-simulation

combine the HSpice response source with Raspberry Pi protocol execution and are not presented as physical hardware latency. Merkle-root generation, pre-flight activation, proof verification, and transcript-hash measurements use the 48-byte TupleHash256 outputs specified by the protocol.

M. SCALABILITY

The number of intervals per root is n , and the interval duration is ΔT , so one root supports a window of $T_{root-window} = n\Delta T$, and m authorized roots give a total pseudonym capacity of $N_{PID} = mn$.

Cost scales predictably with n : Merkle-root construction, pre-flight activation, and resolving the interval index j from a received pseudonym are all linear, $O(n)$, while Merkle proof verification is logarithmic, $O(\log n)$:

$$C_{root} = C_{lookup} = O(n), \quad C_{verify} = O(\log n). \quad (87)$$

The $O(n)$ lookup cost is a fixed cost of every Phase 4 request, paid once per session rather than once per proof, and its constant-time property (Section III-H1) means it cannot be shortened by exiting early without reopening the timing leak it exists to close. Routine broadcast, by contrast, has constant cryptographic size regardless of n : $O_{RID}^{crypto} = O(1)$.

The main scalability trade-off is clear. Larger n reduces renewal frequency and increases the valid operating window per root, but it increases pre-flight activation time and volatile active-state storage. Larger m reduces the frequency of TA renewal but increases nonvolatile storage for signed root records and verifier authorization database size.

N. PERFORMANCE COMPARISON

Table 18 compares the proposed protocol with representative UAV Remote ID and authentication approaches at the design level. The comparison focuses on where cryptographic cost is placed.

The proposed protocol keeps the broadcast path independent of ML-DSA signature size, ML-KEM ciphertext size, and Merkle proof size. Enrollment and renewal carry the root-authorization cost. Pre-flight activation reconstructs the active Merkle tree. On-demand authentication adds the branch-free pseudonym lookup, PUF reconstruction, ML-DSA key regeneration and signing, Merkle verification, and AEAD processing.

TABLE 17. Raspberry Pi software latency and HSpice co-simulation timing results (30 Raspberry Pi trials per software operation).

Operation	Mean latency (ms)	Std. dev. (ms)
HSpice generation of one 1023-challenge, nine-read response dataset	[simulation result]	[simulation result]
Majority voting, BCH reconstruction, and 256-bit extraction	[measure]	[measure]
One Merkle-root generation, $n = 1024$	[measure]	[measure]
One Merkle-root generation, $n = 16384$	[measure]	[measure]
Pre-flight root activation, $n = 1024$	[measure]	[measure]
Pre-flight root activation, $n = 16384$	[measure]	[measure]
Routine broadcast identifier selection	0.004	0.005
Verifier authentication and session setup	0.069	0.012
On-demand authentication, $n = 1024$	[measure]	[measure]
On-demand authentication, $n = 16384$	[measure]	[measure]

TABLE 18. Design-level performance comparison with representative schemes.

Scheme	Routine broadcast burden	Authentication cost placement	Post-quantum support	Stored UAV secret exposure
ARID [17]	Anonymous Remote ID broadcast	Cryptographic protection tied to Remote ID mechanism	No	Uses classical cryptographic material
A2RID [18]	Anonymous Remote ID structures	Direct anonymous authentication	No	Uses classical authentication material
pqRID [19]	Compact pseudonymous broadcast	Post-quantum authentication separated from broadcast	Yes	Uses pre-provisioned cryptographic material and pseudonym certificates
TS-PAID [21]	Not designed for Remote ID broadcast constraints	PUF-based Internet-of-Drones authentication	No	PUF reduces stored-secret exposure
Proposed protocol	Profile P: 16-byte PID in an accepted SSI mapping; Profile S: mandatory RID broadcast unchanged	Merkle proof and PQC material transmitted only during authorized unicast authentication	Yes	No persistent UAV private key, no stored S_1 , S_2 , $Seed_k$, or interval secrets

VI. CONCLUSION

This paper presented a PUF-rooted post-quantum authentication protocol for UAV Remote Identification. The design separates two deployment profiles. Profile P maps a changing 128-bit pseudonym to a Specific Session ID field only where the applicable authority and means of compliance permit that use. Profile S leaves the jurisdiction-approved identifier and complete mandatory Remote ID message set unchanged and supplies the PUF-derived proof as supplementary authorized-verifier authentication. Merkle paths, root authorization records, ML-KEM objects, and ML-DSA signatures are carried only through an authorized unicast exchange.

PUF-derived credential regeneration is the main departure from pqRID and related broadcast-authentication schemes. A majority-voted RO-PUF response vector is stabilized by a syndrome-based fuzzy extractor. Domain separation produces one secret for deterministic ML-DSA identity regeneration and another for Merkle-root and interval-pseudonym derivation. The UAV retains public challenges, helper data, root metadata, and TA-signed records, but not its extracted master secret, private signing key, root seed, interval secrets, or pseudonym table in nonvolatile storage. The empirical evaluation is a circuit-software co-simulation: HSpice supplies modeled 45 nm response data and the Raspberry Pi 5 executes reconstruction and protocol operations. The reported PUF statistics are model-specific and are not evidence of fabricated-silicon entropy, aging behavior, physical-device environmental reliability, or resistance to physical attacks.

During flight, the UAV rebuilds one authorized Merkle tree locally and keeps the active pseudonyms and authentication paths in volatile memory. An authorized verifier signs a request containing the profile-specific observed reference and an ephemeral ML-KEM public key. After certificate, scope, freshness, and revocation checks, the UAV establishes a post-quantum session, releases the corresponding Merkle path, regenerates its ML-DSA key, and signs a transcript bound to the request, root, interval, flight context, nonce, timestamp, and KEM ciphertext. The protocol exposes no interval secret and provides one verifier-checkable, transcript-bound authentication result.

The standards boundary is also explicit. A 16-byte pseudonym fitting inside the ASTM UAS-ID capacity establishes only size compatibility. Profile P requires an externally assigned or accepted SSI subtype, applicable Session-ID semantics, the complete mandatory message set, standards-conforming BLE/Wi-Fi framing, and the relevant regulatory acceptance process. Profile S avoids identifier substitution but does not provide pseudonymous public broadcast. The ML-KEM/ML-DSA exchange is supplementary authorized-verifier authentication and is not represented as DRIP Broadcast Authentication or as an FAA, EU, or Japanese product certification.

The security boundary is explicit. ML-KEM-768 and ML-DSA-65 provide category-3 security for session establishment and signatures, respectively, but the protocol does not claim a uniform category-3 level. Merkle and transcript binding use

384-bit TupleHash outputs; the 128-bit pseudonym remains a compact public identifier with duplicate rejection; and the fuzzy extractor uses the separately stated 2^{-64} statistical-distance target under the simulation-derived entropy bound. A physical deployment must independently establish the required entropy and reconstruction reliability on manufactured devices. Passive disclosure of the UAV's nonvolatile storage reveals no persistent signing key or extracted PUF secret, whereas active control of an intact captured UAV can regenerate the device credentials and must be contained through revocation. The symbolic specification represents transcript authentication, Merkle binding, current revocation state, passive storage disclosure, and verifier-database disclosure under an authenticated confidential enrollment assumption. Identifier-level pseudonymity is supported by the KDF-based construction and prior Remote ID privacy designs, but no formal unlinkability proof is claimed. Crash recovery uses a reserve-before-use journal: an interval pseudonym is never reassigned after restart, although a crash between durable reservation and first transmission may consume an unused index.

REFERENCES

- [1] Belwafi, K., Alkadi, R., Alameri, S. A., Al Hamadi, H., and Shoufan, A., "Unmanned aerial vehicles' remote identification: A tutorial and survey," *IEEE Access*, vol. 10, pp. 87577–87601, 2022.
- [2] Tedeschi, P., Al Nuaimi, F. A., Awad, A. I., and Natalizio, E., "Privacy-aware remote identification for unmanned aerial vehicles: Current solutions, potential threats, and future directions," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 1069–1080, 2023.
- [3] Federal Aviation Administration, "Remote ID Final Rule: Remote Identification of Unmanned Aircraft," 2021. [Online]. Available: https://www.faa.gov/sites/faa.gov/files/2021-08/RemoteID_Final_Rule.pdf
- [4] Electronic Code of Federal Regulations, "14 CFR Part 89—Remote Identification of Unmanned Aircraft," U.S. Government Publishing Office. [Online]. Available: <https://www.ecfr.gov/current/title-14/chapter-I/subchapter-F/part-89>
- [5] ASTM International, "ASTM F3411-22a: Standard Specification for Remote ID and Tracking," Technical Standard, ASTM International, 2022. doi: 10.1520/F3411-22A.
- [6] ASTM International, "ASTM F3586-22: Standard Practice for Remote ID Means of Compliance to FAA Regulation 14 CFR Part 89," Technical Standard, ASTM International, 2022. doi: 10.1520/F3586-22.
- [7] Federal Aviation Administration, "Remote ID for Industry and Standards Bodies," Apr. 17, 2024. [Online]. Available: https://www.faa.gov/uas/getting_started/remote_id/industry
- [8] European Union, "Commission Delegated Regulation (EU) 2019/945 of 12 March 2019 on unmanned aircraft systems and on third-country operators of unmanned aircraft systems," 2019. [Online]. Available: https://eur-lex.europa.eu/eli/reg_del/2019/945/oj
- [9] European Union, "Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the rules and procedures for the operation of unmanned aircraft," 2019. [Online]. Available: https://eur-lex.europa.eu/eli/reg_impl/2019/947/oj
- [10] Ministry of Land, Infrastructure, Transport and Tourism, "Requirements for Remote ID devices and applications," Technical Specification no. 001582250, Civil Aviation Bureau, Japan, 2022. [Online]. Available: <https://www.mlit.go.jp/koku/content/001582250.pdf>
- [11] Shor, P. W., "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997. doi: 10.1137/S009753979529317X.
- [12] Cybersecurity and Infrastructure Security Agency, National Security Agency, and National Institute of Standards and Technology, "Quantum-Readiness: Migration to Post-Quantum Cryptography," Joint Cybersecurity Information Sheet, Aug. 2023. [Online]. Available: <https://www.cisa.gov/resources-tools/resources/quantum-readiness-migration-post-quantum-cryptography>
- [13] National Cyber Security Centre, "Timelines for Migration to Post-Quantum Cryptography," Guidance, NCSC, United Kingdom, Mar. 20, 2025. [Online]. Available: <https://www.ncsc.gov.uk/guidance/pqc-migration-timelines>
- [14] National Institute of Standards and Technology, "FIPS PUB 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," Federal Information Processing Standards Publication, U.S. Department of Commerce, Aug. 2024. doi: 10.6028/NIST.FIPS.203.
- [15] National Institute of Standards and Technology, "FIPS PUB 204: Module-Lattice-Based Digital Signature Standard," Federal Information Processing Standards Publication, U.S. Department of Commerce, Aug. 2024. doi: 10.6028/NIST.FIPS.204.
- [16] National Institute of Standards and Technology, "FIPS PUB 205: Stateless Hash-Based Digital Signature Standard," Federal Information Processing Standards Publication, U.S. Department of Commerce, Aug. 2024. doi: 10.6028/NIST.FIPS.205.
- [17] P. Tedeschi, S. Sciancalepore, and R. Di Pietro, "ARID: Anonymous remote identification of unmanned aerial vehicles," in *Proc. 37th Annu. Comput. Security Appl. Conf. (ACSAC)*, pp. 207–218, 2021, doi: 10.1145/3485832.3485834.
- [18] E. Wisse, P. Tedeschi, S. Sciancalepore, and R. Di Pietro, "A²RID—Anonymous direct authentication and remote identification of commercial drones," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10587–10604, Jun. 2023, doi: 10.1109/JIOT.2023.3240477.
- [19] J. Kim, Y. Lee, J.-H. Kang, and S.-H. Seo, "pqRID: Compact, anonymous remote identification and post-quantum authentication for drones," *IEEE Internet Things J.*, early access, 2026, doi: 10.1109/JIOT.2025.3650221.
- [20] V. Pal, B. S. Acharya, S. Shrivastav, S. Saha, A. Joglekar, and B. Amrutur, "PUF based secure framework for hardware and software security of drones," in *Proc. 2020 Asian Hardware Oriented Security Trust Symp. (AsianHOST)*, pp. 1–6, 2020, doi: 10.1109/Asian-HOST51057.2020.9358264.
- [21] S. A. Chaudhry, A. Irshad, B. A. Alzahrani, A. Alhindi, M. Shariq, and A. K. Das, "TS-PAID: A two-stage PUF-based lightweight authentication protocol for Internet of Drones," *IEEE Access*, vol. 13, pp. 1458–1469, 2025, doi: 10.1109/ACCESS.2024.3523352.
- [22] Zhang, Z., Hsu, C.-H., Au, M. H., Harn, L., Cui, J., Xia, Z., and Zhao, Z., "PRLAP-IoD: A PUF-Based Robust and Lightweight Authentication Protocol for Internet of Drones," *Computer Networks*, vol. 238, p. 110118, 2024, doi: 10.1016/j.comnet.2023.110118.
- [23] S. Card, A. Wiethuechter, R. Moskowitz, and A. Gurtov, "Drone Remote Identification Protocol (DRIP) Requirements and Terminology," RFC 9153, Feb. 2022. doi: 10.17487/RFC9153.
- [24] R. Moskowitz, S. Card, A. Wiethuechter, and A. Gurtov, "DRIP Entity Tag (DET) for Unmanned Aircraft System Remote ID (UAS RID)," RFC 9374, Mar. 2023. doi: 10.17487/RFC9374.
- [25] S. Card, A. Wiethuechter, R. Moskowitz, S. Zhao, and A. Gurtov, "Drone Remote Identification Protocol (DRIP) Architecture," RFC 9434, Jul. 2023. doi: 10.17487/RFC9434.
- [26] A. Wiethuechter, S. Card, and R. Moskowitz, "DRIP Entity Tag (DET) Authentication Formats and Protocols for Broadcast Remote Identification (RID)," RFC 9575, Jun. 2024. doi: 10.17487/RFC9575.
- [27] Brighente, A., Conti, M., and Sciancalepore, S., "Hide and Seek: Privacy-Preserving and FAA-Compliant Drones Location Tracing," in *Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES '22)*, Vienna, Austria, ACM, Aug. 2022, Art. no. 55. doi: 10.1145/3538969.3543784.
- [28] Brighente, A., Conti, M., Schotsman, M., and Sciancalepore, S., "Obfuscated Location Disclosure for Remote ID Enabled Drones," *arXiv preprint arXiv:2407.14256*, 2024.
- [29] Fu, L., Morón, P. T., Queralta, J. P., Hästbacka, D., Edelman, H., and Westerlund, T., "Is Alice Really in Wonderland? UWB-Based Proof of Location for UAVs with Hyperledger Fabric Blockchain," *arXiv preprint arXiv:2304.00795*, 2023.
- [30] Gassend, B., Clarke, D., van Dijk, M., and Devadas, S., "Silicon Physical Random Functions," in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, ACM, pp. 148–160, 2002. doi: 10.1145/586110.586132.
- [31] Suh, G. E., and Devadas, S., "Physical Unclonable Functions for Device Authentication and Secret Key Generation," in *Proceedings of the 44th Annual Design Automation Conference (DAC)*, ACM, pp. 9–14, 2007. doi: 10.1145/1278480.1278484.
- [32] Dodis, Y., Ostrovsky, R., Reyzin, L., and Smith, A., "Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data,"

- SIAM Journal on Computing*, vol. 38, no. 1, pp. 97–139, 2008. doi: 10.1137/060651380.
- [33] Nair, A. S., Thampi, S. M., and Jafeel, V., “A Post-Quantum Secure PUF Based Cross-Domain Authentication Mechanism for Internet of Drones,” *Vehicular Communications*, vol. 47, p. 100780, 2024. doi: 10.1016/j.vehcom.2024.100780.
 - [34] U. Rührmair, J. Sölter, F. Sehnke, X. Xu, A. Mahmoud, V. Stoyanova, G. Dror, J. Schmidhuber, W. Bursleson, and S. Devadas, “PUF modeling attacks on simulated and silicon data,” *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 11, pp. 1876–1891, Nov. 2013, doi: 10.1109/TIFS.2013.2279798.
 - [35] Merkle, R. C., “A Digital Signature Based on a Conventional Encryption Function,” in *Advances in Cryptology — CRYPTO ’87*, Lecture Notes in Computer Science, vol. 293, Springer, pp. 369–378, 1988. doi: 10.1007/3-540-48184-2_32.
 - [36] Turan, M. S., McKay, K., Kang, J. H., Kelsey, J., and Chang, D., “NIST SP 800-232: Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions,” NIST Special Publication, National Institute of Standards and Technology, Aug. 2025. doi: 10.6028/NIST.SP.800-232.
 - [37] C. Dobraunig, M. Eichseder, F. Mendel, and M. Schläffer, “Ascon v1.2: Lightweight authenticated encryption and hashing,” *J. Cryptol.*, vol. 34, art. no. 33, 2021, doi: 10.1007/s00145-021-09398-9.
 - [38] National Institute of Standards and Technology, “SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash,” NIST Special Publication 800-185, Dec. 2016. doi: 10.6028/NIST.SP.800-185.
 - [39] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,” *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.
 - [40] Meier, S., Schmidt, B., Cremers, C., and Basin, D., “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” in *Computer Aided Verification (CAV 2013)*, Lecture Notes in Computer Science, vol. 8044, Springer, pp. 696–701, 2013. doi: 10.1007/978-3-642-39799-8_48.
 - [41] R. del Pino and G. Niot, “Finally! A compact lattice-based threshold signature,” in *Public-Key Cryptography—PKC 2025*, Lecture Notes in Computer Science, vol. 15676, pp. 169–199, 2025, doi: 10.1007/978-3-031-91826-1_6.
 - [42] Dufka, A., Kravtchenko, S., Laud, P., and Snetkov, N., “Trilithium: Efficient and Universally Composable Distributed ML-DSA Signing,” *IACR Cryptology ePrint Archive*, Report 2025/675, 2025.
 - [43] Linux man-pages project, “fsync(2): Synchronize a file’s in-core state with a storage device,” Linux manual page. [Online]. Available: <https://man7.org/linux/man-pages/man2/fsync.2.html>
 - [44] Linux Kernel Documentation, “ext4 General Information.” [Online]. Available: <https://www.kernel.org/doc/html/latest/admin-guide/ext4.html>
 - [45] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle, “Recommendation for the Entropy Sources Used for Random Bit Generation,” NIST Special Publication 800-90B, Jan. 2018. doi: 10.6028/NIST.SP.800-90B.

HALA M. HAMADEH received the B.S. degree in Electrical Engineering from Yarmouk University, Jordan, and the M.S. and Ph.D. degrees in Computer Engineering from Iowa State University, Ames, Iowa.

ABDALLAH M. ALMOMANI received the B.S. degree in mechanical Engineering from Jordan University of Science and Technology, Jordan, and the M.S. and Ph.D. degrees in Aerospace Engineering from Iowa State University, Ames, Iowa.

...